Proyecto Final Modulo Python

Miguel Tello Gómez

Índice

3 Introducción
3 Objetivos
4 Diseño
7 Código fuente
17 Resultados
21 Pruebas
31 Problemas y Soluciones
31 Conclusiones
32 Trabajo futuro
34 Referencias
35 Apéndices

Introducción

Este proyecto presenta un caso de estudio. En este caso de estudio se desarrolla la solución para el problema de gestión de un taller de vehículos de limpieza utilizando las herramientas dadas durante este módulo de Python. Para entender este proyecto hay que entrar en la ficción de este taller. El taller se especializa en el mantenimiento y reparación de una flota de vehículos de limpieza municipal, incluyendo barredoras y vehículos de recogida de residuos. La gestión actual de las órdenes de trabajo, el inventario de piezas, la asignación de mecánicos y el historial de mantenimiento es manual y, por consiguiente, ineficiente, propensa a errores y carece de trazabilidad.

El objetivo principal de este proyecto es implementar un sistema de gestión informatizado para el taller utilizando el lenguaje de programación Python. Este sistema buscará optimizar los procesos críticos, reducir los tiempos de inactividad de los vehículos, mejorar la planificación del mantenimiento y proporcionar informes detallados sobre la operatividad de la flota. La solución propuesta se desarrollará como un paquete que pueda ser usado por aplicaciones de consola, APIs o incluso Jupyter Notebook, lo que facilita su desarrollo y posterior escalabilidad.

Para la realización del proyecto se ha utilizado Python 3.14 en visual studio code.

Objetivos

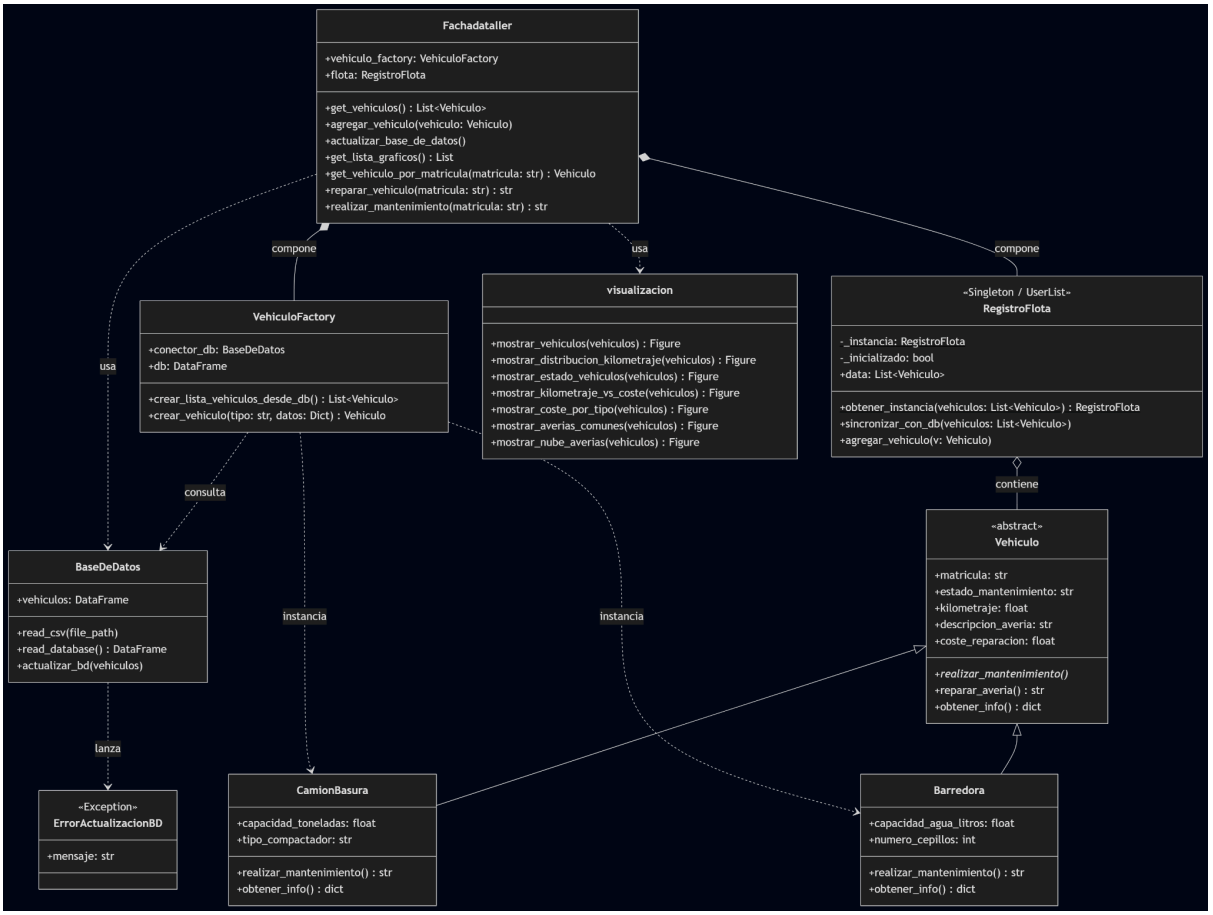
Se ha estructurado el proyecto con el objetivo de satisfacer los siguientes objetivos:

- La capacidad de acceder a un csv con el objetivo de usarlo para simular una base de datos en las pruebas del proyecto.
- La capacidad de realizar gráficas de visualización sobre los datos.

- La capacidad de utilizar orientación a objetos con el propósito de añadir funcionalidades de forma ordenada y claramente visible por medio de UML.

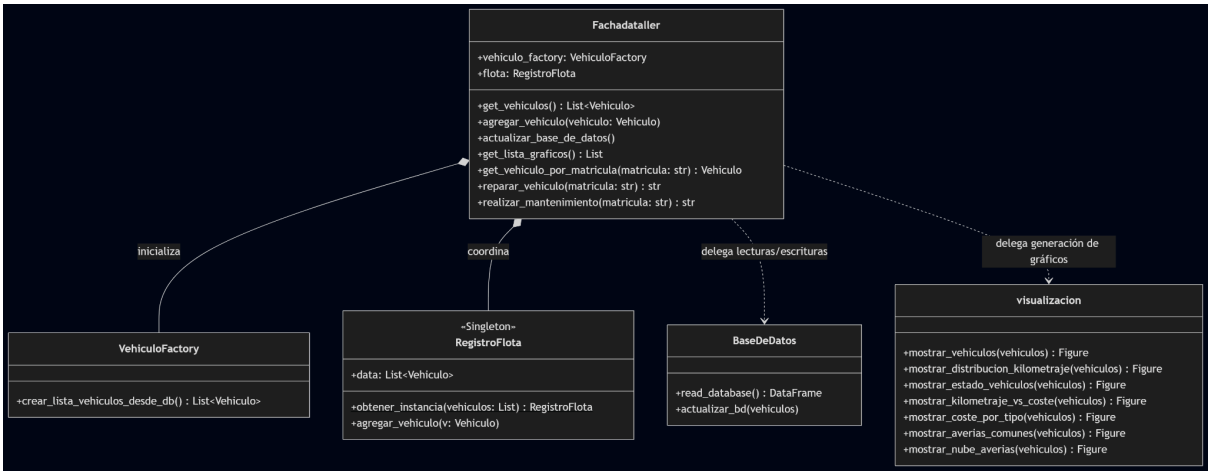
Diseño

El esquema UML del diseño se ha realizado utilizando código mermaid.

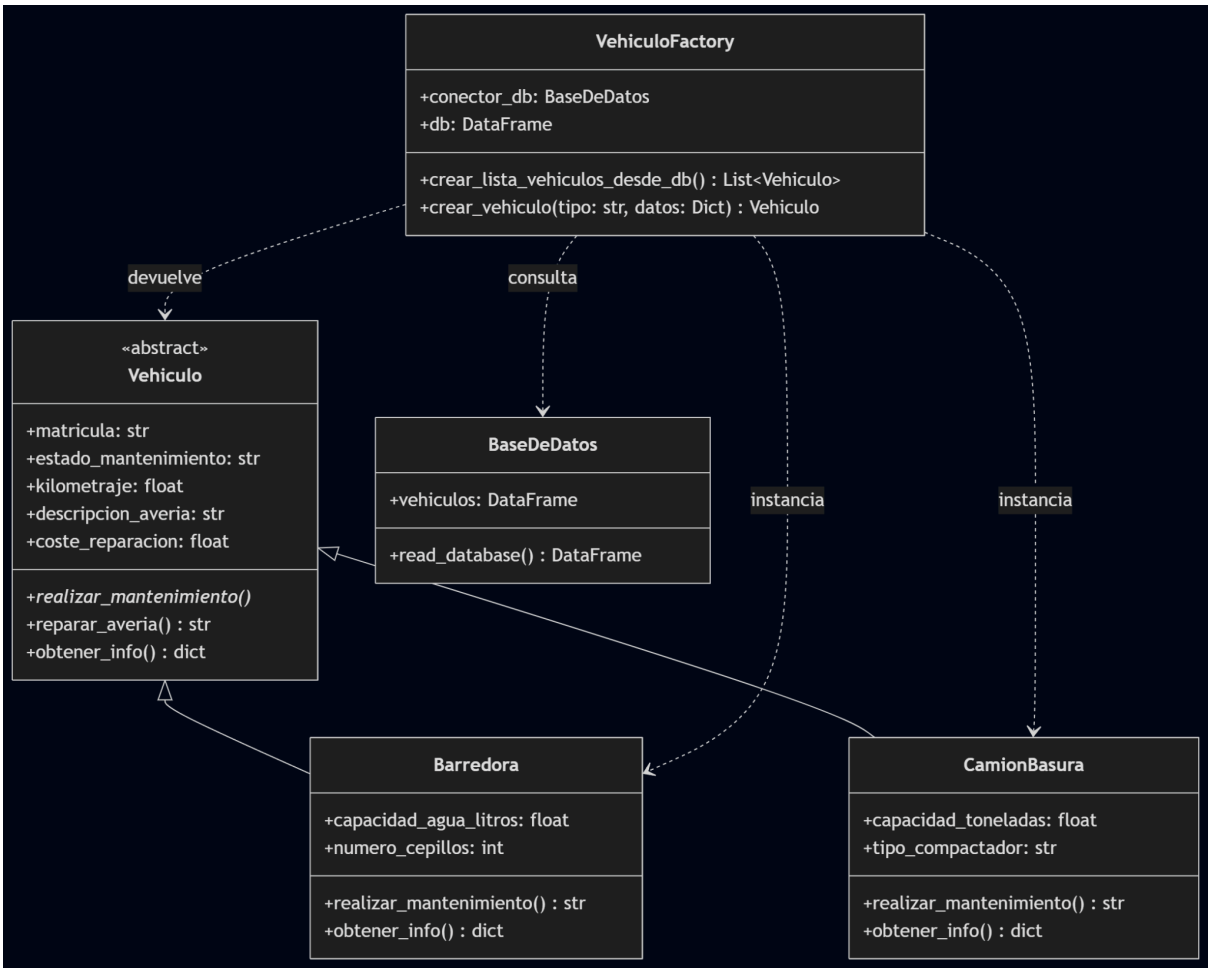


Se han seguido varios patrones de diseño para realizar este proyecto:

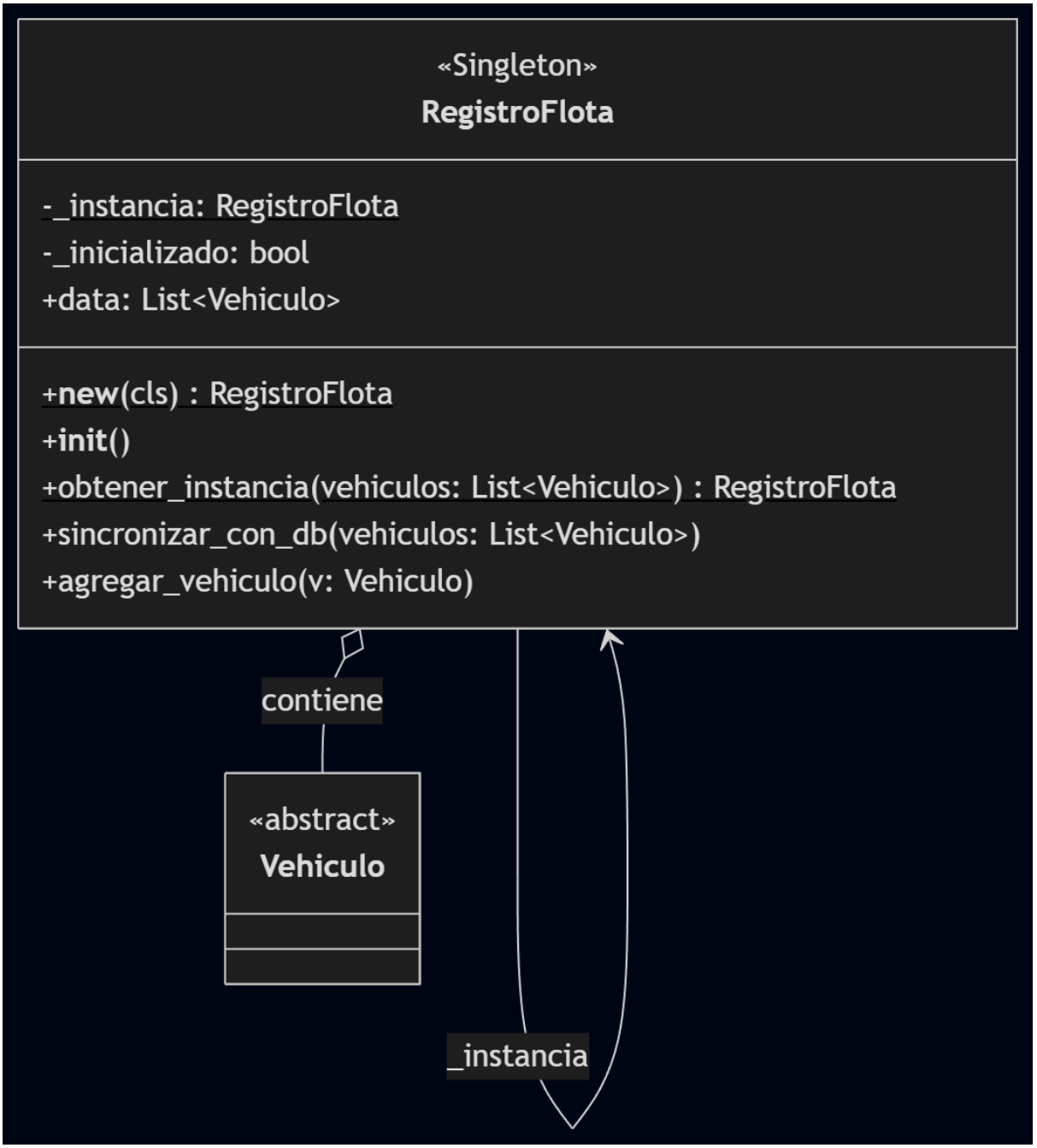
Fachada: es un patrón estructural que ofrece una interfaz simplificada para acceder a un sistema complejo de clases, bibliotecas o frameworks.



Factory: es un patrón de creación que proporciona una interfaz para crear objetos en una superclase, pero permite que las subclases alteren el tipo de objetos que se crearán.



Singleton: es un patrón de creación cuyo objetivo es garantizar que una clase tenga una única instancia en todo el ciclo de vida de la aplicación y proporcionar un punto de acceso global a ella.


«Singleton»
RegistroFlota
-_instancia: RegistroFlota
-_inicializado: bool
+data: List<Vehiculo>
+new(cls) : RegistroFlota
+init()
+obtener_instancia(vehiculos: List<Vehiculo>) : RegistroFlota
+sincronizar_con_db(vehiculos: List<Vehiculo>)
+agregar_vehiculo(v: Vehiculo)
contiene
«abstract»
Vehiculo
_instancia

Código Fuente

Las clases que componen el proyecto son:

Fachadataller: Es la clase principal y la que está expuesta al resto del mundo.

```
class Fachadataller:



    def __init__(self):
        self.vehiculo_factory = VehiculoFactory()
        self.flota =
RegistroFlota.obtener_instancia(self.vehiculo_factory.crear_lista_vehic
ulos_desde_db())



    def get_vehiculos(self):
        return self.flota.data

    def agregar_vehiculo(self, vehiculo):
        self.flota.agregar_vehiculo(vehiculo)

    def actualizar_base_de_datos(self):
        bd = BaseDeDatos()
        bd.actualizar_bd(self.flota.data)


    def get_lista_graficos(self):
        self.lista_vehiculos = pd.DataFrame()
        bd = BaseDeDatos()
        self.lista_vehiculos = bd.read_database()
        vis = visualizacion()
        lista_graficos = []

lista_graficos.append(vis.mostrar_vehiculos(self.lista_vehiculos))

lista_graficos.append(vis.mostrar_distribucion_kilometraje(self.lista_v
ehiculos))
```

```
lista_graficos.append(vis.mostrar_estado_vehiculos(self.lista_vehiculos
))

lista_graficos.append(vis.mostrar_kilometraje_vs_coste(self.lista_vehic
ulos))

lista_graficos.append(vis.mostrar_coste_por_tipo(self.lista_vehiculos))

lista_graficos.append(vis.mostrar_averias_comunes(self.lista_vehiculos)
)

lista_graficos.append(vis.mostrar_nube_averias(self.lista_vehiculos))
        return lista_graficos

    def get_vehiculo_por_matricula(self, matricula: str):
        return next((v for v in self.get_vehiculos() if v.matricula ==
matricula), None)

    def reparar_vehiculo(self, matricula: str):
        vehiculo = self.get_vehiculo_por_matricula(matricula)
        if vehiculo:
            return vehiculo.reparar_averia()
        else:
            return "No se encontró un vehículo con matrícula " +
matricula

    def realizar_mantenimiento(self, matricula: str):
        vehiculo = self.get_vehiculo_por_matricula(matricula)

        if vehiculo:
            return vehiculo.realizar_mantenimiento()
        else:
            return "No se encontró un vehículo con matrícula " +
matricula
```

BasedeDatos: La clase BaseDeDatos se encarga de la gestión y el acceso a los datos del sistema, que se encuentran almacenados de manera "hardcodeada" dentro de un archivo en formato CSV. Esto implica que la ruta y el nombre del archivo de datos están definidos directamente en el código de la clase. Incluye una clase excepción personalizada. Se ha cambiado el csv en el que se actualiza por que es necesario para que el original no se modifique durante las pruebas.

```
class ErrorActualizacionBD(Exception):



    def __init__(self, mensaje):

        mensaje = "Error al actualizar la base de datos: " +
str(mensaje)
        self.mensaje = mensaje
        super().__init__(self.mensaje)

class BaseDeDatos:
    def __init__(self):
        self.vehiculos = pd.DataFrame()

    def read_csv(self, file_path):
        try:
            self.vehiculos = pd.read_csv(file_path)
        except Exception as e:
            print("Error al leer el archivo CSV: " + str(e))

    def read_database(self):
        self.read_csv(ruta_directa)
        return self.vehiculos

    def actualizar_bd(self, vehiculos):
        try:
            ruta2 = os.path.join(os.path.dirname(__file__),
"vehiculos"+str(len(vehiculos))+".csv")
            vehiculos_df = pd.DataFrame([v.obtener_info() for v in
vehiculos])
            vehiculos_df.to_csv(ruta2, index=False)
        except Exception as e:
            raise ErrorActualizacionBD(e)
```

visualizacion: Esta clase se encarga de implementar las funcionalidades de los gráficos.

```
nltk.download('stopwords')
stopwords_spanish = set(stopwords.words('spanish'))

class visualizacion:
    def __init__(self):
        pass



    def mostrar_vehiculos(self, vehiculos):
        tipos = vehiculos['tipo_vehiculo'].value_counts()
        fig =plt.figure(figsize=(8, 5))
        plt.bar(tipos.index, tipos.values, color=['blue', 'orange',
'green'])
        plt.title('Cantidad de Vehículos por Tipo')
        plt.xlabel('Tipo de Vehículo')
        plt.ylabel('Cantidad')
        plt.xticks(rotation=45)
        plt.tight_layout()
        return fig

    def mostrar_distribucion_kilometraje(self, vehiculos):
        fig =plt.figure(figsize=(8, 5))
        plt.hist(vehiculos['kilometraje'].dropna(), bins=20,
color='skyblue', edgecolor='black')
        plt.title('Distribución del Kilometraje de los Vehículos')
        plt.xlabel('Kilometraje')
        plt.ylabel('Frecuencia (Número de vehículos)')
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.tight_layout()
        return fig

    def mostrar_estado_vehiculos(self, vehiculos):
        estados = vehiculos['estado_mantenimiento'].value_counts()
        fig =plt.figure(figsize=(7, 7))
        plt.pie(estados.values, labels=estados.index,
autopct='%1.1f%%', startangle=140,
```

```
                colors=['#ff9999','#66b3ff','#99ff99','#ffcc99'])
        plt.title('Proporción de Vehículos por Estado')
        plt.tight_layout()
        return fig

    def mostrar_kilometraje_vs_coste(self, vehiculos):
        fig =plt.figure(figsize=(8, 5))
        plt.scatter(vehiculos['kilometraje'],
vehiculos['coste_reparacion'], alpha=0.6, color='purple')
        plt.title('Relación entre Kilometraje y Coste de Reparación')
        plt.xlabel('Kilometraje')
        plt.ylabel('Coste de Reparación (€)')
        plt.grid(True, linestyle='--', alpha=0.5)
        plt.tight_layout()
        return fig

    def mostrar_coste_por_tipo(self, vehiculos):
        fig =plt.figure(figsize=(10, 6))
        tipos = vehiculos['tipo_vehiculo'].dropna().unique()
        datos_boxplot = [vehiculos[vehiculos['tipo_vehiculo'] ==
tipo]['coste_reparacion'].dropna() for tipo in tipos]
        plt.boxplot(datos_boxplot, labels=tipos, patch_artist=True,
                    boxprops=dict(facecolor='lightblue', color='blue'))
        plt.title('Distribución del Coste de Reparación por Tipo de
Vehículo')
        plt.xlabel('Tipo de Vehículo')
        plt.ylabel('Coste de Reparación (€)')
        plt.xticks(rotation=45)
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.tight_layout()
        return fig

    def mostrar_averias_comunes(self, vehiculos):
        averias =
vehiculos['descripcion_averia'].value_counts().head(5)
        fig =plt.figure(figsize=(8, 5))
        plt.barh(averias.index, averias.values, color='coral')
        plt.title('Top 5 Averías Más Comunes')
        plt.xlabel('Cantidad de Casos')
        plt.ylabel('Descripción de la Avería')
        plt.gca().invert_yaxis()
        plt.tight_layout()
        return fig
```

```
    def mostrar_nube_averias(self, vehiculos):

        stopwords = set(WordCloud().stopwords.union({"vehículo",
"matrícula", "kilometraje", "coste", "reparación", "mantenimiento",
"avería", "falla", "problema", "daño"}))
        texto_averias = "
".join(vehiculos['descripcion_averia'].dropna().astype(str))
        stopwords.update(stopwords_spanish)
        nube = WordCloud(width=800, height=400,
                         background_color='white',
                         colormap='viridis',
                         max_words=100,
                         stopwords=stopwords).generate(texto_averias)
        fig =plt.figure(figsize=(10, 5))
        plt.imshow(nube, interpolation='bilinear')
        plt.axis('off')
        plt.title('Términos más frecuentes en Descripción de Averías',
fontsize=14)
        plt.tight_layout()
        return fig
```

singleton: Esta clase implementa el patrón Singleton. Permite acceder a una única instancia de la estructura de datos que funciona como la lista de vehículos. La estructura de datos utilizada es UserList de la libreria collections de python

```
class RegistroFlota(UserList):

    _instancia = None

    def __new__(cls, *args, **kwargs):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia

    def __init__(self):
```

```
        if not hasattr(self, '_inicializado'):
            super().__init__()
            self._inicializado = True

    @classmethod
    def obtener_instancia(cls, vehiculos: List[Vehiculo]) ->
'RegistroFlota':
        if cls._instancia is None:
            cls._instancia = cls()
        cls._instancia.sincronizar_con_db(vehiculos)
        return cls._instancia

    def sincronizar_con_db(self, vehiculos: List[Vehiculo]):
        self.data = vehiculos

    def agregar_vehiculo(self, v: Vehiculo):
        self.append(v)
```

vehiculosfactory: Implementación del patrón de diseño factory utilizada habitualmente para aprovechar el polimorfismo. No permite hacer clases de objetos que no existen.

```
class VehiculoFactory:
    def __init__(self):
        pass

    def crear_lista_vehiculos_desde_db(self):
        self.conector_db = BaseDeDatos()
        self.db = self.conector_db.read_database()
        vehiculos = []
        for _, fila in self.db.iterrows():
            tipo = fila['tipo_vehiculo']
            datos = {
                "matricula": fila['matricula'],
                "estado_mantenimiento": fila['estado_mantenimiento'],
                "kilometraje": fila['kilometraje'],
                "descripcion_averia": fila['descripcion_averia'],
                "coste_reparacion": fila['coste_reparacion']
            }
            if tipo == "Camion Basura":
                datos.update({
```

```
                    "capacidad_toneladas": 12,
                    "tipo_compactador": "Compactador de tornillo"
                })
            elif tipo == "Barredora":
                datos.update({
                    "capacidad_agua_litros": 5000,
                    "numero_cepillos": 4
                })
            else:
                print("Tipo de vehículo desconocido en la base de
datos:" + tipo)
                continue

            vehiculo = self.crear_vehiculo(tipo, datos)
            vehiculos.append(vehiculo)
        return vehiculos




    def crear_vehiculo(self, tipo: str, datos: Dict[str, Any]) ->
Vehiculo:
        if tipo == "Camion Basura":
            return CamionBasura(**datos)
        elif tipo == "Barredora":
            return Barredora(**datos)
        else:
            raise ValueError("Tipo de vehículo desconocido: " + tipo)
```

vehiculo: Clase abstracta de la que heredan los distintos vehículos.

```
class Vehiculo(ABC):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float, descripcion_averia: str = "", coste_reparacion:
float = 0.0):
        self.matricula = matricula
        self.estado_mantenimiento = estado_mantenimiento
        self.kilometraje = kilometraje
        self.descripcion_averia = descripcion_averia
```

```
        self.coste_reparacion = coste_reparacion

    @abstractmethod
    def realizar_mantenimiento(self):
        pass

    def reparar_averia(self):

        self.estado_mantenimiento = "Reparado"
        return "Vehículo " + self.matricula + " reparado. Descripción
de la avería: " + self.descripcion_averia + ". Coste de reparación: " +
str(self.coste_reparacion) + "."

    def obtener_info(self):
        tipo = self.__class__.__name__
        tipo = ''.join([' ' + c if c.isupper() else c for c in
tipo]).strip()


        return {
            "matricula": self.matricula,
            "estado_mantenimiento": self.estado_mantenimiento,
            "kilometraje": self.kilometraje,
            "tipo_vehiculo": tipo,
            "descripcion_averia": self.descripcion_averia,
            "coste_reparacion": self.coste_reparacion
        }
```

barredora: primer tipo de vehiculo

```
class Barredora(Vehiculo):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float,
                 capacidad_agua_litros: float, numero_cepillos: int,
descripcion_averia: str = "", coste_reparacion: float = 0.0):
        super().__init__(matricula, estado_mantenimiento, kilometraje,
descripcion_averia, coste_reparacion)
        self.capacidad_agua_litros = capacidad_agua_litros
        self.numero_cepillos = numero_cepillos

    def realizar_mantenimiento(self):
```

```
        return ("[Barredora] Limpiando " + str(self.numero_cepillos) +
" cepillos y revisando bomba de agua en vehículo " + self.matricula +
".")

    def obtener_info(self):
        info = super().obtener_info()
        info.update({
            "capacidad_agua_litros": self.capacidad_agua_litros,
            "numero_cepillos": self.numero_cepillos
        })
        return info
```

camionBasura: segundo tipo de vehículo

```
class CamionBasura(Vehiculo):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float,
                 capacidad_toneladas: float, tipo_compactador: str,
descripcion_averia: str = "", coste_reparacion: float = 0.0):
        super().__init__(matricula, estado_mantenimiento, kilometraje,
descripcion_averia, coste_reparacion)
        self.capacidad_toneladas = capacidad_toneladas
        self.tipo_compactador = tipo_compactador

    def realizar_mantenimiento(self):
        return ("[Camión Basura] Mantenimiento preventivo del
compactador " + self.tipo_compactador + " en vehículo " +
self.matricula + ".")



    def obtener_info(self):
        info = super().obtener_info()
        info.update({
            "capacidad_toneladas": self.capacidad_toneladas,
            "tipo_compactador": self.tipo_compactador
        })
        return info
```

Resultados

El csv de pruebas es así:

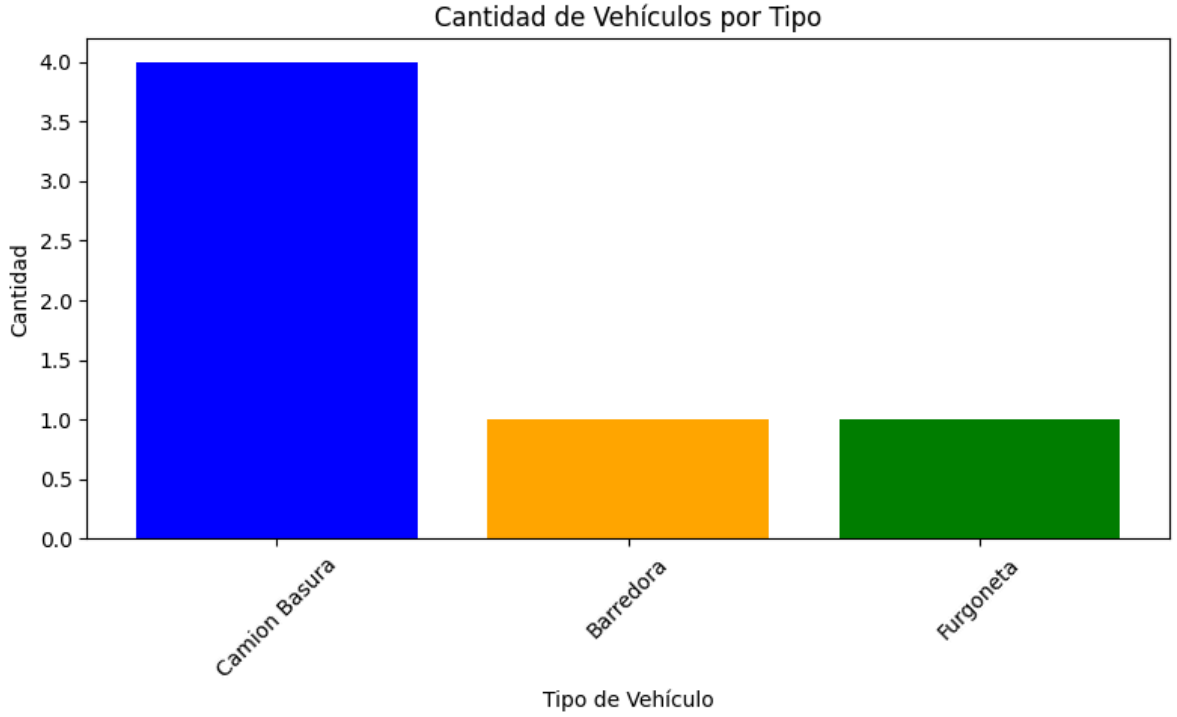
cei_template > cei_template > vehiculos.csv

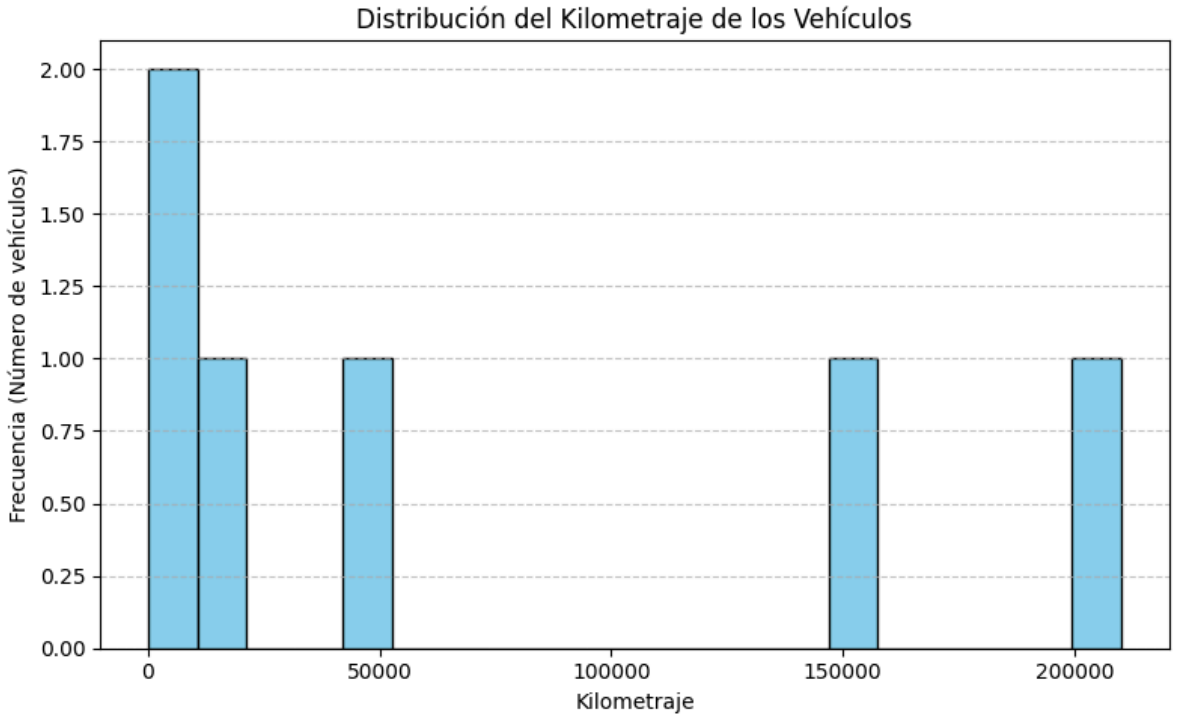
	matricula	estado_mantenimiento	kilometraje	tipo_vehiculo	capacidad_toneladas	tipo_compactador	capacidad_agua_litros	numero_cepillos	descripcion_averia	coste_reparacion
1	ABC123	Pendiente	150200	Camion Basura	12.0	Compactador de tornillo			Cambio de aceite	100
2	BAB654	En curso	12000	Barredora			5000.0	4.0	Cambio de aceite	111
3	IJH456	Pendiente	210000	Camion Basura	12.0	Compactador de tornillo			Rueda pinchada, Cambio de aceite	112
4	XXX069	En curso	45000	Camion Basura	12.0	Compactador de tornillo			Cristal roto	1223
5	NNN333	Bueno	5000	Camion Basura	10.0	Compactador de tornillo			Cristal dañado	1334
6	PPP368	En curso	100	Furgoneta	10.0				Puerta rota, Cambio de aceite, Cristal Roto	33333
7										

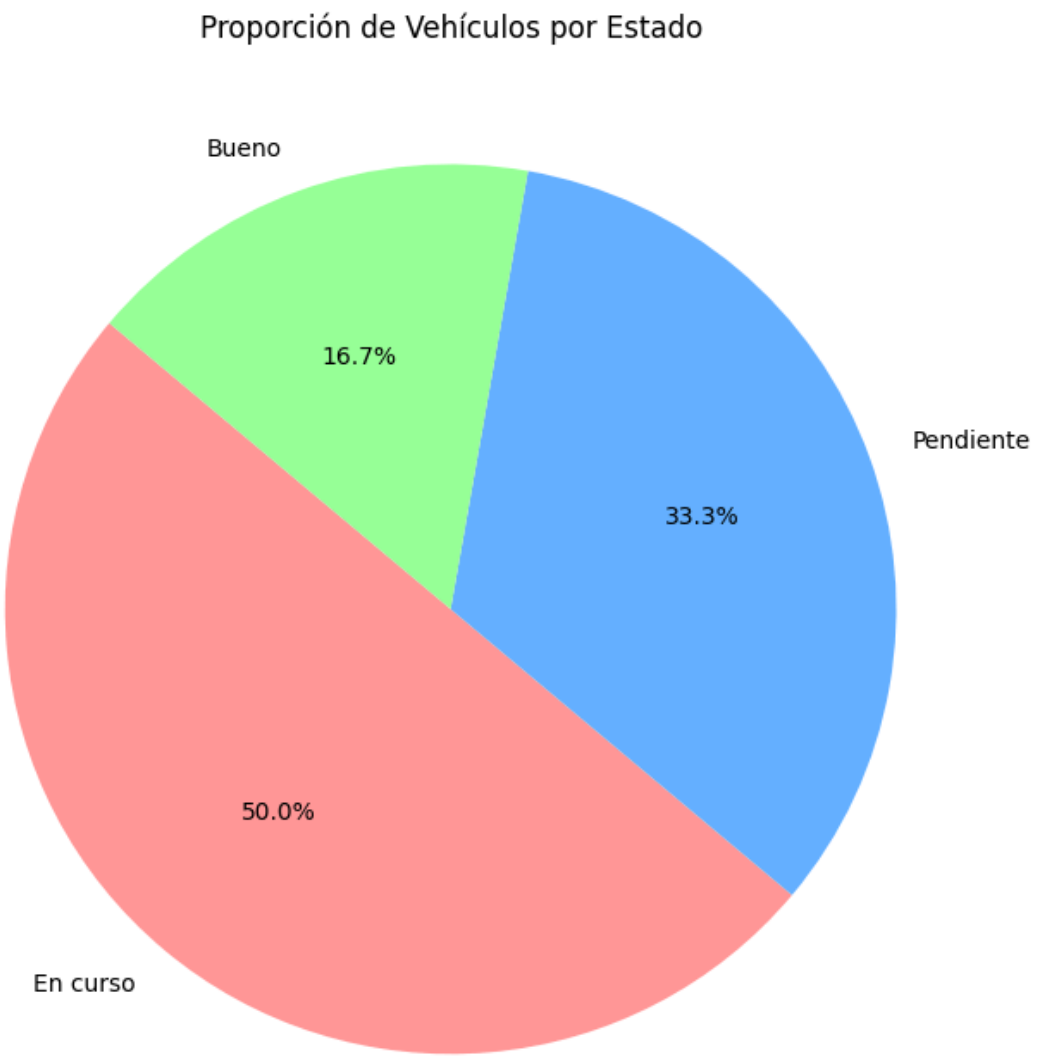
Los campos son:

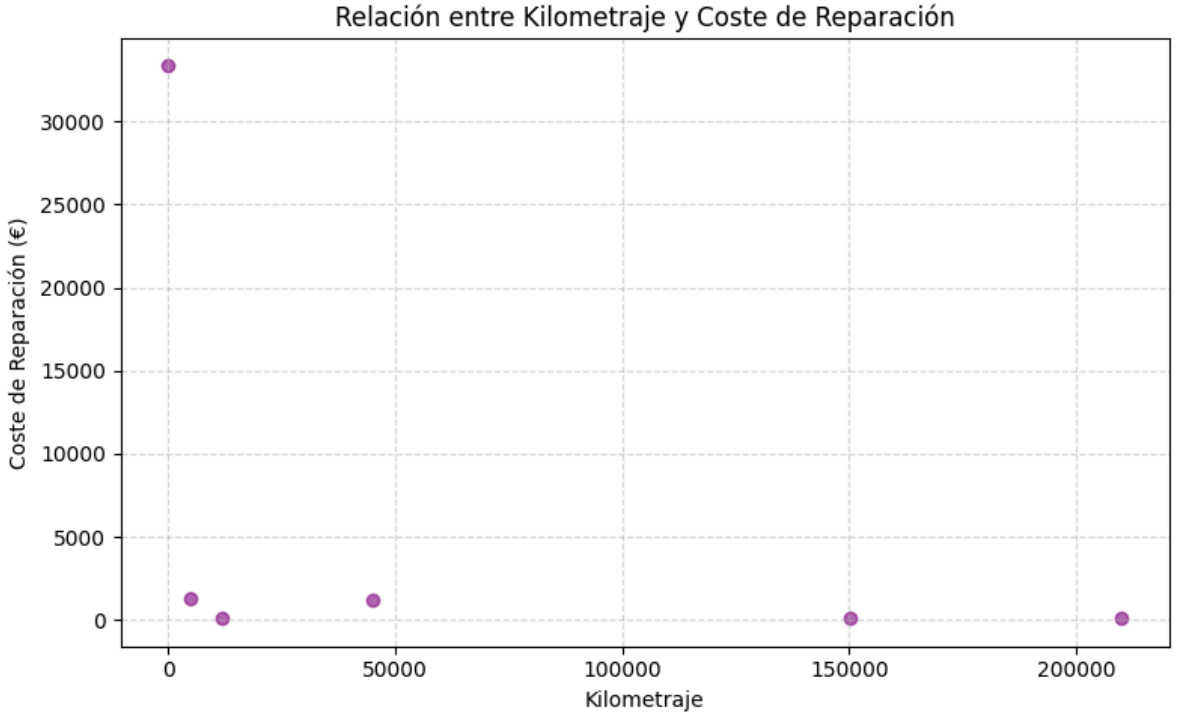
- Matricula: dependiendo de la base de datos que se use podría ser el id.
- estado_mantenimiento: evalúa el progreso del mantenimiento
- kilometraje: número de kilómetros recorridos.
- tipo_vehiculo: determina el tipo de vehículo. La clase factory elimina las filas que no sean de las clases correctas.
- capacidad_toneladas/tipo_compactador/capacidad_agua_litros/numero_cepillos: campos que estan disponibles solo en algunos de los vehículos.
- descripcion_averia: una descripción en formato texto diseñada para hacerse análisis de palabras.
- coste_reparacion: el que sería mayormente el campo objetivo en cálculos de regresiones.

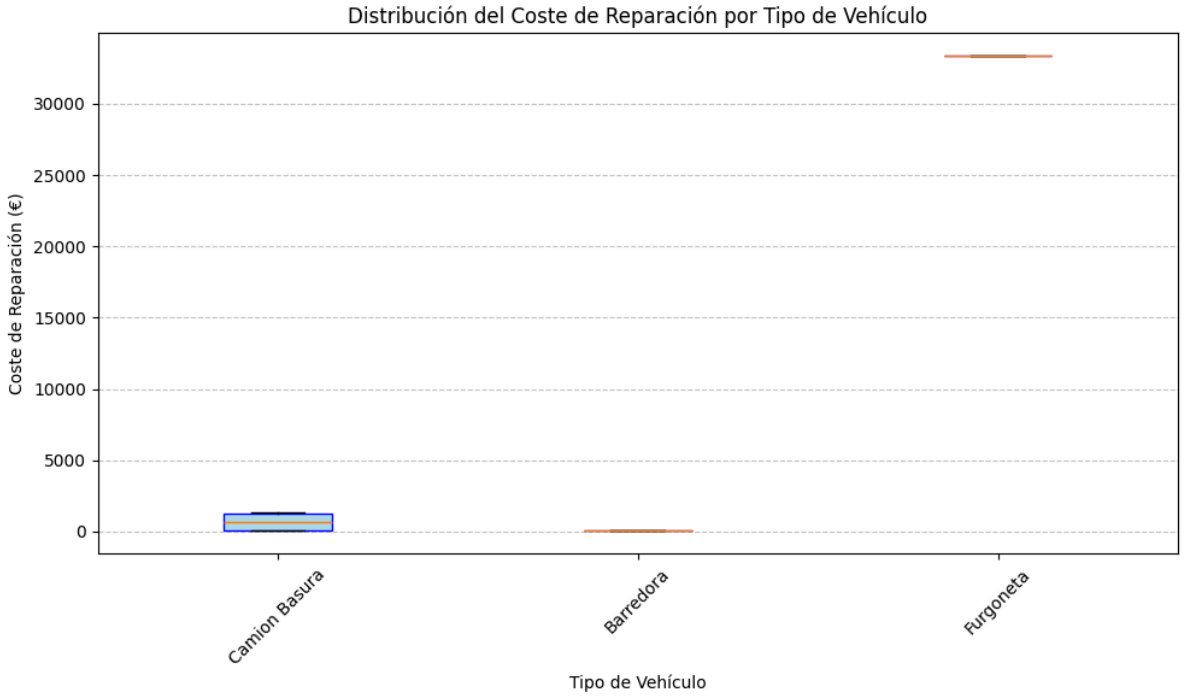
Las gráficas son así:

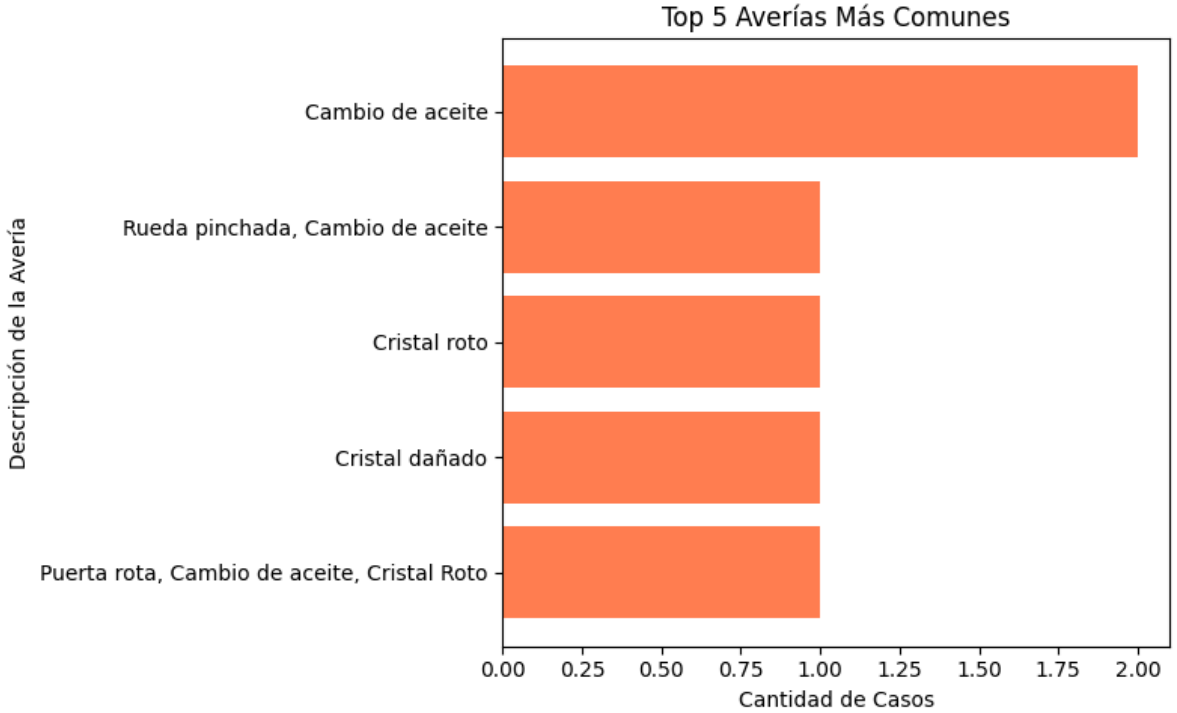

Cantidad de Vehículos por Tipo
Cantidad
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
0.0
Camion Basura
Barredora
Furgoneta
Tipo de Vehículo

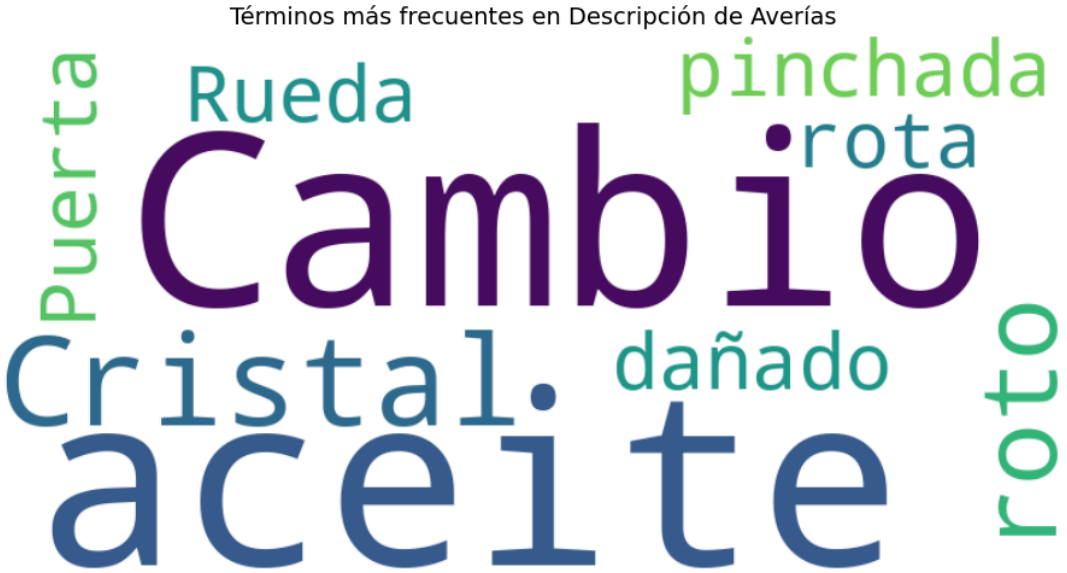

Distribución del Kilometraje de los Vehículos
Frecuencia (Número de vehículos)
2.00
1.75
1.50
1.25
1.00
0.75
0.50
0.25
0.00
0
50000
100000
150000
200000
Kilometraje


Proporción de Vehículos por Estado
Bueno
16.7%
Pendiente
33.3%
50.0%
En curso


Relación entre Kilometraje y Coste de Reparación
Coste de Reparación (€)
30000
25000
20000
15000
10000
5000
0
0
50000
100000
150000
200000
Kilometraje


Distribución del Coste de Reparación por Tipo de Vehículo
Coste de Reparación (€)
30000
25000
20000
15000
10000
5000
0
Camion Basura
Barredora
Furgoneta
Tipo de Vehículo





Pruebas

Se han utilizado las siguientes pruebas:

script main:

```
import os
import sys
from cei_template.Fachadataller import Fachadataller

sys.path.append(os.path.abspath("cei_template"))

def main():

    taller = Fachadataller()
    print(taller.get_vehiculos())
    for i, grafico in enumerate(taller.get_lista_graficos()):
        grafico.show()
        grafico.savefig(os.path.join(os.path.dirname(__file__),
"graficos", "imagen_" + str(i) + ".png"))

    print(taller.realizar_mantenimiento("ABC123"))
    print(taller.realizar_mantenimiento("BAB654"))

taller.agregar_vehiculo(taller.vehiculo_factory.crear_vehiculo("Camion
Basura", {
        "matricula": "NEW999",
        "estado_mantenimiento": "Bueno",
        "kilometraje": 5000,
        "capacidad_toneladas": 10,
        "tipo_compactador": "Compactador de tornillo"
    }))
    taller.actualizar_base_de_datos()
    print([vehiculo.obtener_info() for vehiculo in
taller.get_vehiculos()])
    taller.reparar_vehiculo("NEW999")
    print(taller.get_vehiculo_por_matricula("NEW999").obtener_info())




if __name__ == '__main__':
    main()
```

tkinter

```
import tkinter as tk
from tkinter import ttk, messagebox
import os
import sys
from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
import pandas as pd
from cei_template.Fachadataller import Fachadataller

sys.path.append(os.path.abspath("cei_template"))


class AppTaller:
    def __init__(self, root):
        self.root = root
        self.root.title("Gestión Avanzada de Taller")
        self.root.geometry("900x650")

        try:
            self.fachada = Fachadataller()
        except Exception as e:
            messagebox.showerror("Error", f"Error al inicializar la
base de datos: {e}")
            self.root.destroy()
            return

        self.tabs = ttk.Notebook(self.root)
        self.tab_tabla = ttk.Frame(self.tabs)
        self.tab_graficos = ttk.Frame(self.tabs)

        self.tabs.add(self.tab_tabla, text="Gestión de Flota")
        self.tabs.add(self.tab_graficos, text="Panel de Estadísticas")
        self.tabs.pack(expand=1, fill="both")


        self.lista_figuras = []

        self.construir_tab_tabla()
        self.construir_tab_graficos()
```

```
        self.cargar_tabla()

    def construir_tab_tabla(self):
        # -- Frame para la tabla --
        frame_arriba = ttk.Frame(self.tab_tabla, padding=10)
        frame_arriba.pack(fill=tk.BOTH, expand=True)

        self.tree = ttk.Treeview(frame_arriba, show='headings')
        scrollbar_y = ttk.Scrollbar(frame_arriba, orient=tk.VERTICAL,
command=self.tree.yview)
        self.tree.configure(yscroll=scrollbar_y.set)

        scrollbar_y.pack(side=tk.RIGHT, fill=tk.Y)
        self.tree.pack(side=tk.LEFT, fill=tk.BOTH, expand=True)

        frame_acciones = ttk.LabelFrame(self.tab_tabla, text="Acciones
Rápidas", padding=10)
        frame_acciones.pack(fill=tk.X, padx=10, pady=10)

        ttk.Label(frame_acciones, text="Matrícula:").pack(side=tk.LEFT,
padx=(0, 5))
        self.entry_matricula = ttk.Entry(frame_acciones, width=15)
        self.entry_matricula.pack(side=tk.LEFT, padx=(0, 15))

        btn_reparar = ttk.Button(frame_acciones, text="Reparar Avería",
command=self.accion_reparar)
        btn_reparar.pack(side=tk.LEFT, padx=5)

        btn_mantenimiento = ttk.Button(frame_acciones,
text="Mantenimiento", command=self.accion_mantenimiento)
        btn_mantenimiento.pack(side=tk.LEFT, padx=5)

        btn_actualizar = ttk.Button(frame_acciones, text="Guardar en
BD", command=self.accion_actualizar_bd)
        btn_actualizar.pack(side=tk.RIGHT, padx=5)

    def cargar_tabla(self):
        for item in self.tree.get_children():
            self.tree.delete(item)

        vehiculos = self.fachada.get_vehiculos()
        if not vehiculos:
```

```
            return

        if isinstance(vehiculos, list) and hasattr(vehiculos[0],
'__dict__'):
            columnas = list(vars(vehiculos[0]).keys())
            filas = [list(vars(v).values()) for v in vehiculos]

        elif isinstance(vehiculos, pd.DataFrame):
            columnas = list(vehiculos.columns)
            filas = vehiculos.values.tolist()
        else:
            return

        self.tree["columns"] = columnas
        for col in columnas:
            self.tree.heading(col, text=str(col).upper())
            self.tree.column(col, width=100, anchor=tk.CENTER)

        for fila in filas:
            self.tree.insert("", tk.END, values=fila)

    def accion_reparar(self):
        matricula = self.entry_matricula.get().strip()
        if not matricula:
            messagebox.showwarning("Atención", "Introduce una
matrícula.")
            return
        resultado = self.fachada.reparar_vehiculo(matricula)
        messagebox.showinfo("Resultado", resultado)
        self.cargar_tabla()

    def accion_mantenimiento(self):
        matricula = self.entry_matricula.get().strip()
        if not matricula:
            messagebox.showwarning("Atención", "Introduce una
matrícula.")
            return
        resultado = self.fachada.realizar_mantenimiento(matricula)
        messagebox.showinfo("Resultado", resultado)
        self.cargar_tabla()

    def accion_actualizar_bd(self):
        try:
```

```
            self.fachada.actualizar_base_de_datos()
            messagebox.showinfo("Éxito", "Base de datos actualizada
correctamente.")
        except Exception as e:
            messagebox.showerror("Error", f"No se pudo actualizar la
BD: {e}")

    def construir_tab_graficos(self):
        frame_controles = ttk.Frame(self.tab_graficos, padding=10)
        frame_controles.pack(fill=tk.X)

        ttk.Label(frame_controles, text="Selecciona un
gráfico:").pack(side=tk.LEFT, padx=5)

        opciones_graficos = [
            "1. Cantidad de Vehículos",
            "2. Distribución de Kilometraje",
            "3. Estado de los Vehículos",
            "4. Kilometraje vs Coste",
            "5. Coste por Tipo de Vehículo",
            "6. Averías Más Comunes",
            "7. Nube de Averías"
        ]

        self.combo_graficos = ttk.Combobox(frame_controles,
values=opciones_graficos, state="readonly", width=35)
        self.combo_graficos.pack(side=tk.LEFT, padx=5)
        self.combo_graficos.bind("<<ComboboxSelected>>",
self.cambiar_grafico)

        btn_cargar = ttk.Button(frame_controles, text="Generar/Recargar
Gráficos", command=self.cargar_graficos)
        btn_cargar.pack(side=tk.LEFT, padx=15)

        self.frame_canvas = ttk.Frame(self.tab_graficos)
        self.frame_canvas.pack(fill=tk.BOTH, expand=True, padx=10,
pady=10)

    def cargar_graficos(self):
        try:
            self.lista_figuras = self.fachada.get_lista_graficos()
            self.combo_graficos.current(0)
            self.cambiar_grafico(None)
```

```
        except Exception as e:
            messagebox.showerror("Error", f"Error al generar gráficos:
{e}")

    def cambiar_grafico(self, event):
        if not self.lista_figuras:
            return
        indice = self.combo_graficos.current()
        figura = self.lista_figuras[indice]
        for widget in self.frame_canvas.winfo_children():
            widget.destroy()
        if figura:
            canvas = FigureCanvasTkAgg(figura,
master=self.frame_canvas)
            canvas.draw()
            canvas.get_tk_widget().pack(fill=tk.BOTH, expand=True)
        else:
            ttk.Label(self.frame_canvas, text="No se pudo generar este
gráfico.").pack(pady=20)

if __name__ == "__main__":
    root = tk.Tk()
    app = AppTaller(root)
    root.mainloop()
```

fastapi

```
from http.client import HTTPException
import io

from fastapi import FastAPI, Response
from fastapi.responses import JSONResponse
from cei_template.Fachadataller import Fachadataller
import os
import sys

sys.path.append(os.path.abspath("cei_template"))
app = FastAPI()

taller = Fachadataller()

@app.get("/")
def home():
```

```
    return {"mensaje": "API de Gestión de Taller activa"}

@app.get("/vehiculos")
def obtener_vehiculos():
    """Retorna la lista de vehículos en formato JSON"""
    df = taller.get_vehiculos()

    datos = df.to_dict(orient="records")
    return JSONResponse(content=datos)

@app.get("/grafico/{grafico_id}")
def obtener_grafico(grafico_id: int):
    lista_graficos = taller.get_lista_graficos()
    if grafico_id < 0 or grafico_id >= len(lista_graficos):
        raise HTTPException(status_code=404, detail="Gráfico no
encontrado en la lista")
    buf = io.BytesIO()
    lista_graficos[grafico_id].savefig(buf, format='png')
    imagen_bytes = buf.getvalue()


    return Response(content=imagen_bytes, media_type="image/png")

if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="127.0.0.1", port=8000)
```

fastapi+streamlit: Para correr streamlit se utiliza el comando: streamlit run mainstreamlit.py

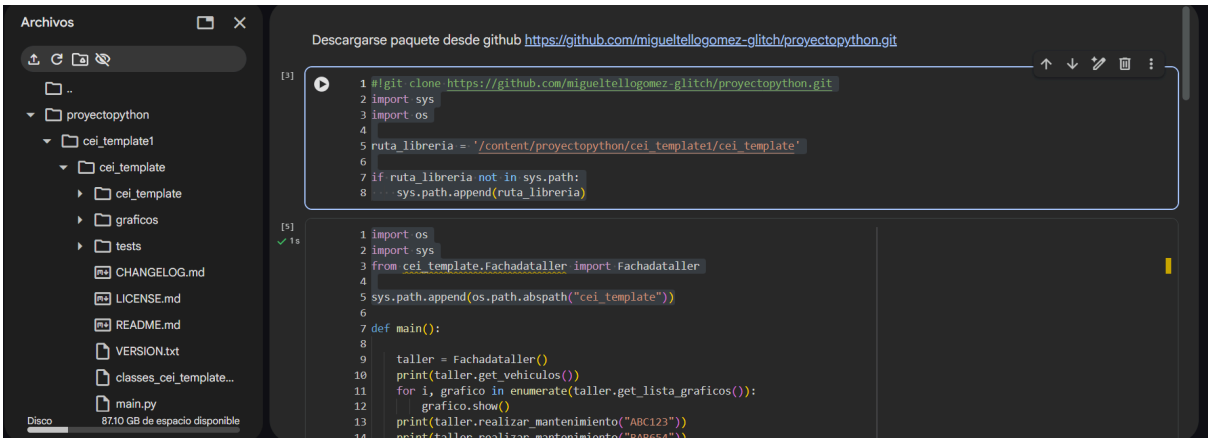
```
import streamlit as st
import requests
import pandas as pd
from io import BytesIO

#comando: streamlit run mainstreamlit.py
API_URL = "http://127.0.0.1:8000"

st.set_page_config(page_title="Gestión de Taller", layout="wide")
```

```
st.title("🚗 Panel de Control - Fachada Taller")

st.sidebar.header("Opciones")
menu = st.sidebar.radio("Ir a:", ["Ver Inventario", "Visualizaciones"])

if menu == "Ver Inventario":
    st.subheader("Listado de Vehículos")

    if st.button("Cargar/Actualizar Datos"):
        try:
            response = requests.get(f"{API_URL}/vehiculos")

            if response.status_code == 200:
                data = response.json()
                if data:
                    df = pd.DataFrame(data)
                    st.dataframe(df, use_container_width=True)
                    st.success(f"Se han cargado {len(df)} vehículos.")
                else:
                    st.warning("La base de datos está vacía.")
            else:
                st.error("Error al obtener datos de la API.")
        except Exception as e:
            st.error(f"No se pudo conectar con la API: {e}")

elif menu == "Visualizaciones":
    st.subheader("Análisis Gráfico")

    grafico_id = st.number_input("Seleccione el ID del gráfico",
min_value=0, max_value=10, value=0, step=1)


    if st.button("Mostrar Gráfico"):
        try:
            with st.spinner(f"Cargando gráfico {grafico_id}..."):
                response =
requests.get(f"{API_URL}/grafico/{grafico_id}")

            if response.status_code == 200:
                imagen_bytes = BytesIO(response.content)
                st.image(
                    imagen_bytes,
                    caption=f"Gráfico ID: {grafico_id}",
```

```
                    use_container_width=True
                )
            elif response.status_code == 404:
                st.warning(f"No se encontró ningún gráfico con el ID
{grafico_id}.")
            else:
                st.error(f"Error {response.status_code}: No se pudo
obtener el gráfico.")

        except Exception as e:
            st.error(f"Error de conexión con la API: {e}")

st.sidebar.info("Conectado a FastAPI Backend")
```

Google Colab: para descargar el paquete desde github usa
!git clone https://github.com/migueltellogomez-glitch/proyectopython.git

```
#!git clone
https://github.com/migueltellogomez-glitch/proyectopython.git
import sys
import os

ruta_libreria = '/content/proyectopython/cei_template1/cei_template'

if ruta_libreria not in sys.path:
    sys.path.append(ruta_libreria)
```



Problemas y Soluciones

El desarrollo de este proyecto se llevó a cabo sin mayores dificultades, transcurriendo de manera fluida. El principal desafío que se presentó fue el manejo de la librería Tkinter, dado que no fue cubierta durante el curso y requirió una investigación.

Conclusiones

Este proyecto ha logrado implementar con éxito un sistema de gestión informatizado para el taller de vehículos de limpieza.

Logros Clave:

1. Uso Efectivo de Patrones de Diseño: Se han aplicado correctamente los patrones Fachada, Factory y Singleton. La Fachadataller proporciona una interfaz simplificada para el usuario, la VehiculoFactory gestiona la creación polimórfica de objetos Barredora y CamionBasura, y el RegistroFlota asegura una única instancia central de la lista de vehículos (Singleton), facilitando la coherencia de los datos.
2. Modularidad y Escalabilidad: El diseño orientado a objetos y la división en módulos (BaseDeDatos, visualizacion, clases de Vehículo) permiten una fácil extensión y mantenimiento del código. El sistema se diseñó como un paquete Python, lo que valida su potencial de escalabilidad para diferentes tipos de aplicaciones.
3. Visualización de Datos: Se ha integrado la capacidad de generar gráficos informativos utilizando matplotlib y wordcloud, lo que proporciona al taller una herramienta valiosa para el análisis de la flota, el kilometraje, los costes de reparación y las averías más comunes.
4. Despliegue Multiplataforma: La solución ha sido demostrada funcionando en cuatro entornos diferentes:
    - Consola (script main): Para lógica de negocio básica y operaciones por lotes.

    - Aplicación de Escritorio (Tkinter): Proporcionando una interfaz gráfica interactiva para la gestión diaria.
    - API Web (FastAPI + Streamlit): Demostrando cómo el sistema puede exponer sus funcionalidades (datos y gráficos) a través de una API RESTful para ser consumido por un frontend web moderno (Streamlit).

Desafíos Superados:

El principal reto técnico fue la integración de la librería Tkinter, ya que no formaba parte del temario del módulo, lo cual se resolvió mediante investigación individual, logrando una interfaz gráfica funcional para la manipulación de los datos del taller.

En resumen, el proyecto no solo ha satisfecho los requisitos funcionales iniciales, sino que también ha demostrado una sólida aplicación de los principios de la programación orientada a objetos (POO) y de patrones de diseño, resultando en una solución robusta y lista para futuras expansiones.

Trabajo futuro

Estas son mis sugerencias detalladas para la futura evolución y mejora de este proyecto:

Infraestructura y Despliegue (Escalabilidad y Disponibilidad):

- Utilizar una plataforma en la nube para alojar la API (Backend as a Service - BaaS): Para garantizar alta disponibilidad, escalabilidad dinámica y un despliegue robusto y profesional, se recomienda migrar el alojamiento de la API a servicios en la nube. Opciones como AWS (Amazon Web Services) con servicios como EC2 o Lambda (para una arquitectura serverless), Google Cloud Platform (GCP) con Compute Engine o Cloud Functions, o Microsoft Azure con App Service o Functions son ideales. Esto permitiría gestionar

picos de tráfico de manera eficiente y reduciría la carga operativa. Se sugiere configurar un Continuous Integration/Continuous Deployment (CI/CD) para automatizar las actualizaciones.

Gestión de Datos (Persistencia y Rendimiento):

- Usar SQL o una base de datos NoSQL como MongoDB para la persistencia de datos: El uso de estructuras de datos persistentes es crucial.
    - Bases de Datos Relacionales (SQL): Si la estructura de los datos es rígida y se requiere alta integridad transaccional (ACID), una base de datos SQL como PostgreSQL o MySQL sería la elección adecuada.
    - Bases de Datos NoSQL (Ej. MongoDB): Si se prevé una evolución rápida del esquema de datos o se necesita una alta velocidad de lectura/escritura y escalabilidad horizontal (distribución de la base de datos), se recomienda utilizar una base de datos NoSQL basada en documentos como MongoDB.
- Usar Apache Spark para el procesamiento de grandes DataFrames: A medida que la cantidad de datos crezca, las capacidades nativas de procesamiento de datos en Python podrían volverse ineficientes. Integrar Apache Spark (posiblemente a través de PySpark) permitiría la creación de DataFrames distribuidos, lo que facilitaría el procesamiento masivo de datos, la ejecución de análisis complejos y la aplicación de modelos de Machine Learning a gran escala.

Funcionalidades y Contenido:

- Ampliar funcionalidades y vehículos:
    - Funcionalidades Adicionales: Se podría integrar un módulo de predicción de fallos basado en los datos recopilados, un sistema de optimización de rutas basado en el tráfico en tiempo real, o una interfaz de visualización de datos más interactiva para el usuario final (utilizando herramientas como Plotly Dash o Streamlit).
    - Ampliación del Catálogo de Vehículos: Introducir más tipos de vehículos con diferentes parámetros y modelos de

consumo (e.g., motocicletas, camiones de carga pesada, vehículos eléctricos con modelado de batería y autonomía) para hacer el proyecto más versátil y aplicable a un espectro más amplio de casos de uso.
    - Integración Externa: Explorar la posibilidad de integrar la API con servicios externos de mapas (Google Maps API, OpenStreetMap) para enriquecer los datos de ubicación y tráfico.

Referencias

- Interprete de código mermaid: https://mermaid.ai/web
- Documentacion libreria collections:

  https://docs.python.org/es/3.9/library/collections.html
- Libro patrones de diseño:

  https://refactoring.guru/files/design-patterns-es-demo.pdf
- Documentación oficial de FastAPI: https://fastapi.tiangolo.com/es/
- Documentación oficial de Streamlit: https://docs.streamlit.io/
- Guía de Matplotlib: https://matplotlib.org/stable/users/index.html
- Documentación de WordCloud para Python:

  <https://amueller.github.io/word_cloud/index.html>
- Repositorio Git versión 1:

  https://github.com/migueltellogomez-glitch/proyectoceipython.git
- Repositorio Git version 2:

  https://github.com/migueltellogomez-glitch/proyectopython.git

- Guia tkinter: https://guia-tkinter.readthedocs.io/es/develop/

Apéndice

UML en código mermaid

```
classDiagram
    %% Definición de Clases Base y Derivadas (Herencia)
    class Vehiculo {
        <<abstract>>
        +matricula: str
        +estado_mantenimiento: str
        +kilometraje: float
        +descripcion_averia: str
        +coste_reparacion: float
        +realizar_mantenimiento()*
        +reparar_averia() str
        +obtener_info() dict
    }

    class Barredora {
        +capacidad_agua_litros: float
        +numero_cepillos: int
        +realizar_mantenimiento() str
        +obtener_info() dict
    }

    class CamionBasura {
        +capacidad_toneladas: float
        +tipo_compactador: str
        +realizar_mantenimiento() str
```

```
    +obtener_info() dict
}

Vehiculo <|-- Barredora
Vehiculo <|-- CamionBasura

%% Base de Datos y Excepciones
class BaseDeDatos {
    +vehiculos: DataFrame
    +read_csv(file_path)
    +read_database() DataFrame
    +actualizar_bd(vehiculos)
}

class ErrorActualizacionBD {
    <<Exception>>
    +mensaje: str
}

BaseDeDatos ..> ErrorActualizacionBD : lanza

%% Singleton y Colección
class RegistroFlota {
    <<Singleton / UserList>>
    -_instancia: RegistroFlota$
    -_inicializado: bool
    +data: List~Vehiculo~
    +obtener_instancia(vehiculos: List~Vehiculo~)$ RegistroFlota
    +sincronizar_con_db(vehiculos: List~Vehiculo~)
    +agregar_vehiculo(v: Vehiculo)
}

RegistroFlota o-- Vehiculo : contiene

%% Factory (Patrón de Diseño)
class VehiculoFactory {
    +conector_db: BaseDeDatos
```

```
    +db: DataFrame
    +crear_lista_vehiculos_desde_db() List~Vehiculo~
    +crear_vehiculo(tipo: str, datos: Dict) Vehiculo
}

VehiculoFactory ..> BaseDeDatos : consulta
VehiculoFactory ..> CamionBasura : instancia
VehiculoFactory ..> Barredora : instancia

%% Visualización
class visualizacion {
    +mostrar_vehiculos(vehiculos) Figure
    +mostrar_distribucion_kilometraje(vehiculos) Figure
    +mostrar_estado_vehiculos(vehiculos) Figure
    +mostrar_kilometraje_vs_coste(vehiculos) Figure
    +mostrar_coste_por_tipo(vehiculos) Figure
    +mostrar_averias_comunes(vehiculos) Figure
    +mostrar_nube_averias(vehiculos) Figure
}

%% Fachada (Patrón de Diseño Principal)
class Fachadataller {
    +vehiculo_factory: VehiculoFactory
    +flota: RegistroFlota
    +get_vehiculos() List~Vehiculo~
    +agregar_vehiculo(vehiculo: Vehiculo)
    +actualizar_base_de_datos()
    +get_lista_graficos() List
    +get_vehiculo_por_matricula(matricula: str) Vehiculo
    +reparar_vehiculo(matricula: str) str
    +realizar_mantenimiento(matricula: str) str
}

Fachadataller *-- VehiculoFactory : compone
Fachadataller *-- RegistroFlota : compone
Fachadataller ..> BaseDeDatos : usa
Fachadataller ..> visualizacion : usa
```

Código Python completo:

```
from cei_template.vehiculo import Vehiculo

class Barredora(Vehiculo):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float,
                 capacidad_agua_litros: float, numero_cepillos: int,
descripcion_averia: str = "", coste_reparacion: float = 0.0):
        super().__init__(matricula, estado_mantenimiento, kilometraje,
descripcion_averia, coste_reparacion)
        self.capacidad_agua_litros = capacidad_agua_litros
        self.numero_cepillos = numero_cepillos

    def realizar_mantenimiento(self):
        return ("[Barredora] Limpiando " + str(self.numero_cepillos) + "
cepillos y revisando bomba de agua en vehÃculo " + self.matricula + ".")

    def obtener_info(self):
        info = super().obtener_info()
        info.update({
            "capacidad_agua_litros": self.capacidad_agua_litros,
            "numero_cepillos": self.numero_cepillos
        })
        return info
import pandas as pd
import os

ruta_directa = os.path.join(os.path.dirname(__file__), "vehiculos.csv")


class ErrorActualizacionBD(Exception):



    def __init__(self, mensaje):
```

```
        mensaje = "Error al actualizar la base de datos: " + str(mensaje)
        self.mensaje = mensaje
        super().__init__(self.mensaje)

class BaseDeDatos:
    def __init__(self):
        self.vehiculos = pd.DataFrame()

    def read_csv(self, file_path):
        try:
            self.vehiculos = pd.read_csv(file_path)
        except Exception as e:
            print("Error al leer el archivo CSV: " + str(e))

    def read_database(self):
        self.read_csv(ruta_directa)
        return self.vehiculos

    def actualizar_bd(self, vehiculos):
        try:
            ruta2 = os.path.join(os.path.dirname(__file__),
"vehiculos"+str(len(vehiculos))+".csv")
            vehiculos_df = pd.DataFrame([v.obtener_info() for v in
vehiculos])
            vehiculos_df.to_csv(ruta2, index=False)
        except Exception as e:
            raise ErrorActualizacionBD(e)
from cei_template.vehiculo import Vehiculo


class CamionBasura(Vehiculo):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float,
                 capacidad_toneladas: float, tipo_compactador: str,
descripcion_averia: str = "", coste_reparacion: float = 0.0):
        super().__init__(matricula, estado_mantenimiento, kilometraje,
descripcion_averia, coste_reparacion)
```

```
        self.capacidad_toneladas = capacidad_toneladas
        self.tipo_compactador = tipo_compactador

    def realizar_mantenimiento(self):
        return ("[CamiÃ³n Basura] Mantenimiento preventivo del
compactador " + self.tipo_compactador + " en vehÃculo " +
self.matricula + ".")



    def obtener_info(self):
        info = super().obtener_info()
        info.update({
            "capacidad_toneladas": self.capacidad_toneladas,
            "tipo_compactador": self.tipo_compactador
        })
        return info
from cei_template.basedeDatos import BaseDeDatos
from cei_template.visualizacion import visualizacion
from cei_template.singleton import RegistroFlota
from cei_template.VehiculosFactory import VehiculoFactory
import pandas as pd


class Fachadataller:



    def __init__(self):
        self.vehiculo_factory = VehiculoFactory()
        self.flota =
RegistroFlota.obtener_instancia(self.vehiculo_factory.crear_lista_vehicul
os_desde_db())



    def get_vehiculos(self):
```

```
        return self.flota.data

    def agregar_vehiculo(self, vehiculo):
        self.flota.agregar_vehiculo(vehiculo)

    def actualizar_base_de_datos(self):
        bd = BaseDeDatos()
        bd.actualizar_bd(self.flota.data)


    def get_lista_graficos(self):
        self.lista_vehiculos = pd.DataFrame()
        bd = BaseDeDatos()
        self.lista_vehiculos = bd.read_database()
        vis = visualizacion()
        lista_graficos = []
        lista_graficos.append(vis.mostrar_vehiculos(self.lista_vehiculos))

lista_graficos.append(vis.mostrar_distribucion_kilometraje(self.lista_vehi
culos))

lista_graficos.append(vis.mostrar_estado_vehiculos(self.lista_vehiculos)
)

lista_graficos.append(vis.mostrar_kilometraje_vs_coste(self.lista_vehicul
os))

lista_graficos.append(vis.mostrar_coste_por_tipo(self.lista_vehiculos))

lista_graficos.append(vis.mostrar_averias_comunes(self.lista_vehiculos)
)

lista_graficos.append(vis.mostrar_nube_averias(self.lista_vehiculos))
        return lista_graficos

    def get_vehiculo_por_matricula(self, matricula: str):
```

```
        return next((v for v in self.get_vehiculos() if v.matricula ==
matricula), None)

    def reparar_vehiculo(self, matricula: str):
        vehiculo = self.get_vehiculo_por_matricula(matricula)
        if vehiculo:
            return vehiculo.reparar_averia()
        else:
            return "No se encontrÃ³ un vehÃculo con matrÃcula " + matricula

    def realizar_mantenimiento(self, matricula: str):
        vehiculo = self.get_vehiculo_por_matricula(matricula)

        if vehiculo:
            return vehiculo.realizar_mantenimiento()
        else:
            return "No se encontrÃ³ un vehÃculo con matrÃcula " + matricula


from typing import List
from cei_template.vehiculo import Vehiculo
from collections import UserList

class RegistroFlota(UserList):

    _instancia = None

    def __new__(cls, *args, **kwargs):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
        return cls._instancia

    def __init__(self):
        if not hasattr(self, '_inicializado'):
            super().__init__()
            self._inicializado = True
```

```
    @classmethod
    def obtener_instancia(cls, vehiculos: List[Vehiculo]) -> 'RegistroFlota':
        if cls._instancia is None:
            cls._instancia = cls()
        cls._instancia.sincronizar_con_db(vehiculos)
        return cls._instancia

    def sincronizar_con_db(self, vehiculos: List[Vehiculo]):
        self.data = vehiculos

    def agregar_vehiculo(self, v: Vehiculo):
        self.append(v)
from abc import ABC, abstractmethod

class Vehiculo(ABC):
    def __init__(self, matricula: str, estado_mantenimiento: str,
kilometraje: float, descripcion_averia: str = "", coste_reparacion: float =
0.0):
        self.matricula = matricula
        self.estado_mantenimiento = estado_mantenimiento
        self.kilometraje = kilometraje
        self.descripcion_averia = descripcion_averia
        self.coste_reparacion = coste_reparacion

    @abstractmethod
    def realizar_mantenimiento(self):
        pass

    def reparar_averia(self):

        self.estado_mantenimiento = "Reparado"
        return "VehÃculo " + self.matricula + " reparado. DescripciÃ³n de la
averÃa: " + self.descripcion_averia + ". Coste de reparaciÃ³n: " +
str(self.coste_reparacion) + "."

    def obtener_info(self):
        tipo = self.__class__.__name__
```

```
        tipo = ''.join([' ' + c if c.isupper() else c for c in tipo]).strip()


        return {
            "matricula": self.matricula,
            "estado_mantenimiento": self.estado_mantenimiento,
            "kilometraje": self.kilometraje,
            "tipo_vehiculo": tipo,
            "descripcion_averia": self.descripcion_averia,
            "coste_reparacion": self.coste_reparacion
        }
from typing import List, Dict, Any
from cei_template.vehiculo import Vehiculo
from cei_template.CamionBasura import CamionBasura
from cei_template.Barredora import Barredora
from cei_template.basedeDatos import BaseDeDatos
import pandas as pd

class VehiculoFactory:
    def __init__(self):
        pass

    def crear_lista_vehiculos_desde_db(self):
        self.conector_db = BaseDeDatos()
        self.db = self.conector_db.read_database()
        vehiculos = []
        for _, fila in self.db.iterrows():
            tipo = fila['tipo_vehiculo']
            datos = {
                "matricula": fila['matricula'],
                "estado_mantenimiento": fila['estado_mantenimiento'],
                "kilometraje": fila['kilometraje'],
                "descripcion_averia": fila['descripcion_averia'],
                "coste_reparacion": fila['coste_reparacion']
            }
            if tipo == "Camion Basura":
                datos.update({
```

```
                "capacidad_toneladas": 12,
                "tipo_compactador": "Compactador de tornillo"
            })
        elif tipo == "Barredora":
            datos.update({
                "capacidad_agua_litros": 5000,
                "numero_cepillos": 4
            })
        else:
            print("Tipo de vehÃculo desconocido en la base de datos:" +
tipo)
            continue

        vehiculo = self.crear_vehiculo(tipo, datos)
        vehiculos.append(vehiculo)
    return vehiculos




def crear_vehiculo(self, tipo: str, datos: Dict[str, Any]) -> Vehiculo:
    if tipo == "Camion Basura":
        return CamionBasura(**datos)
    elif tipo == "Barredora":
        return Barredora(**datos)
    else:
        raise ValueError("Tipo de vehÃculo desconocido: " + tipo)

import pandas as pd
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import nltk
from nltk.corpus import stopwords

nltk.download('stopwords')
stopwords_spanish = set(stopwords.words('spanish'))
```

```
class visualizacion:
    def __init__(self):
        pass



    def mostrar_vehiculos(self, vehiculos):
        tipos = vehiculos['tipo_vehiculo'].value_counts()
        fig =plt.figure(figsize=(8, 5))
        plt.bar(tipos.index, tipos.values, color=['blue', 'orange', 'green'])
        plt.title('Cantidad de VehÃculos por Tipo')
        plt.xlabel('Tipo de VehÃculo')
        plt.ylabel('Cantidad')
        plt.xticks(rotation=45)
        plt.tight_layout()
        return fig

    def mostrar_distribucion_kilometraje(self, vehiculos):
        fig =plt.figure(figsize=(8, 5))
        plt.hist(vehiculos['kilometraje'].dropna(), bins=20, color='skyblue',
edgecolor='black')
        plt.title('DistribuciÃ³n del Kilometraje de los VehÃculos')
        plt.xlabel('Kilometraje')
        plt.ylabel('Frecuencia (NÃºmero de vehÃculos)')
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.tight_layout()
        return fig

    def mostrar_estado_vehiculos(self, vehiculos):
        estados = vehiculos['estado_mantenimiento'].value_counts()
        fig =plt.figure(figsize=(7, 7))
        plt.pie(estados.values, labels=estados.index, autopct='%1.1f%%',
startangle=140,
                colors=['#ff9999','#66b3ff','#99ff99','#ffcc99'])
        plt.title('ProporciÃ³n de VehÃculos por Estado')
        plt.tight_layout()
        return fig
```

```
    def mostrar_kilometraje_vs_coste(self, vehiculos):
        fig =plt.figure(figsize=(8, 5))
        plt.scatter(vehiculos['kilometraje'], vehiculos['coste_reparacion'],
alpha=0.6, color='purple')
        plt.title('RelaciÃ³n entre Kilometraje y Coste de ReparaciÃ³n')
        plt.xlabel('Kilometraje')
        plt.ylabel('Coste de ReparaciÃ³n (â,¬)')
        plt.grid(True, linestyle='--', alpha=0.5)
        plt.tight_layout()
        return fig

    def mostrar_coste_por_tipo(self, vehiculos):
        fig =plt.figure(figsize=(10, 6))
        tipos = vehiculos['tipo_vehiculo'].dropna().unique()
        datos_boxplot = [vehiculos[vehiculos['tipo_vehiculo'] ==
tipo]['coste_reparacion'].dropna() for tipo in tipos]
        plt.boxplot(datos_boxplot, labels=tipos, patch_artist=True,
                    boxprops=dict(facecolor='lightblue', color='blue'))
        plt.title('DistribuciÃ³n del Coste de ReparaciÃ³n por Tipo de
VehÃculo')
        plt.xlabel('Tipo de VehÃculo')
        plt.ylabel('Coste de ReparaciÃ³n (â,¬)')
        plt.xticks(rotation=45)
        plt.grid(axis='y', linestyle='--', alpha=0.7)
        plt.tight_layout()
        return fig

    def mostrar_averias_comunes(self, vehiculos):
        averias = vehiculos['descripcion_averia'].value_counts().head(5)
        fig =plt.figure(figsize=(8, 5))
        plt.barh(averias.index, averias.values, color='coral')
        plt.title('Top 5 AverÃas MÃ¡s Comunes')
        plt.xlabel('Cantidad de Casos')
        plt.ylabel('DescripciÃ³n de la AverÃa')
        plt.gca().invert_yaxis()
        plt.tight_layout()
```

```
        return fig


    def mostrar_nube_averias(self, vehiculos):

        stopwords = set(WordCloud().stopwords.union({"vehÃculo",
"matrÃcula", "kilometraje", "coste", "reparaciÃ³n", "mantenimiento",
"averÃa", "falla", "problema", "daÃ±o"}))
        texto_averias = "
".join(vehiculos['descripcion_averia'].dropna().astype(str))
        stopwords.update(stopwords_spanish)
        nube = WordCloud(width=800, height=400,
                         background_color='white',
                         colormap='viridis',
                         max_words=100,
                         stopwords=stopwords).generate(texto_averias)
        fig =plt.figure(figsize=(10, 5))
        plt.imshow(nube, interpolation='bilinear')
        plt.axis('off')
        plt.title('TÃ©rminos mÃ¡s frecuentes en DescripciÃ³n de AverÃas',
fontsize=14)
        plt.tight_layout()
        return fig
"""Top-level package for cei_template."""

__author__ = "Miguel Tello"
__email__ = "miguel.tello.gomez@alumnos.cei.es"
__version__ = "2.0.0"

PACKAGE_NAME = "cei_template"
```

Requirements.txt

```
alabaster @ file:///home/ktietz/src/ci/alabaster_1611921544520/work
altair==5.5.0
anaconda-client==1.11.0
```

```
anaconda-navigator==2.3.1
anaconda-project @
file:///C:/Windows/TEMP/abs_91fu4tfkih/croots/recipe/anaconda-project_1
660339890874/work
annotated-doc==0.0.4
annotated-types==0.7.0
anyio==4.12.1
appdirs==1.4.4
argon2-cffi @
file:///opt/conda/conda-bld/argon2-cffi_1645000214183/work
argon2-cffi-bindings @
file:///C:/ci/argon2-cffi-bindings_1644551690056/work
arrow @ file:///opt/conda/conda-bld/arrow_1649166651673/work
astroid @
file:///C:/Windows/TEMP/abs_b0dtxgpicv/croots/recipe/astroid_1659023126
745/work
astropy @ file:///C:/ci/astropy_1657719656942/work
atomicwrites==1.4.0
attrs @ file:///opt/conda/conda-bld/attrs_1642510447205/work
Automat @ file:///tmp/build/80754af9/automat_1600298431173/work
autopep8 @ file:///opt/conda/conda-bld/autopep8_1650463822033/work
Babel @ file:///tmp/build/80754af9/babel_1620871417480/work
backcall @ file:///home/ktietz/src/ci/backcall_1611930011877/work
backports.functools-lru-cache @
file:///tmp/build/80754af9/backports.functools_lru_cache_1618170165463/
work
backports.tempfile @
file:///home/linux1/recipes/ci/backports.tempfile_1610991236607/work
backports.weakref==1.0.post1
bcrypt @
file:///C:/Windows/Temp/abs_36kl66t_aw/croots/recipe/bcrypt_16595543340
50/work
beautifulsoup4 @ file:///C:/ci/beautifulsoup4_1650293025093/work
binaryornot @ file:///tmp/build/80754af9/binaryornot_1617751525010/work
bitarray @ file:///C:/ci/bitarray_1657729621682/work
bkcharts==0.2
black @ file:///C:/ci/black_1660239974023/work
bleach @ file:///opt/conda/conda-bld/bleach_1641577558959/work
blinker==1.9.0
bokeh @
file:///C:/Windows/TEMP/abs_4a259bc2-ed05-4a1f-808e-ac712cc0900cddqp8sp
7/croots/recipe/bokeh_1658136660686/work
```

```
boto3 @
file:///C:/Windows/TEMP/abs_4009c406-44ba-4406-8996-204d9b11202flt4kglb
k/croots/recipe/boto3_1657820114895/work
botocore @ file:///C:/ci/botocore_1657735875454/work
Bottleneck @
file:///C:/Windows/Temp/abs_3198ca53-903d-42fd-87b4-03e6d03a8381yfwsuve
8/croots/recipe/bottleneck_1657175565403/work
brotlipy==0.7.0
cachetools==6.2.6
certifi @
file:///C:/b/abs_4f5wo627a3/croots/recipe/certifi_1663615677642/work/ce
rtifi
cffi @
file:///C:/Windows/Temp/abs_6808y9x40v/croots/recipe/cffi_1659598653989
/work
chardet @ file:///C:/ci/chardet_1607706937985/work
charset-normalizer @
file:///tmp/build/80754af9/charset-normalizer_1630003229654/work
click @ file:///C:/ci/click_1646038595831/work
cloudpickle @ file:///tmp/build/80754af9/cloudpickle_1632508026186/work
clyent==1.2.2
colorama @
file:///C:/Windows/TEMP/abs_9439aeb1-0254-449a-96f7-33ab5eb17fc8apleb4y
n/croots/recipe/colorama_1657009099097/work
colorcet @ file:///C:/ci/colorcet_1651851676912/work
comtypes==1.1.10
conda==22.9.0
conda-build==3.22.0
conda-content-trust @
file:///C:/Windows/TEMP/abs_4589313d-fc62-4ccc-81c0-b801b4449e833j1ajrw
u/croots/recipe/conda-content-trust_1658126379362/work
conda-pack @ file:///tmp/build/80754af9/conda-pack_1611163042455/work
conda-package-handling @
file:///C:/b/abs_81m11h_i4r/croots/recipe/conda-package-handling_166359
8470202/work
conda-repo-cli==1.0.20
conda-token @
file:///Users/paulyim/miniconda3/envs/c3i/conda-bld/conda-token_1662660
369760/work
conda-verify==3.4.2
constantly==15.1.0
cookiecutter @
file:///opt/conda/conda-bld/cookiecutter_1649151442564/work
```

```
cryptography @ file:///C:/ci/cryptography_1652083563162/work
cssselect==1.1.0
cycler @ file:///tmp/build/80754af9/cycler_1637851556182/work
Cython @
file:///C:/b/abs_0438epndxm/croots/recipe/cython_1663692771227/work
cytoolz==0.11.0
daal4py==2021.6.0
dask @ file:///C:/ci/dask-core_1658515307198/work
datashader @
file:///C:/Windows/TEMP/abs_62cyd2dpuf/croots/recipe/datashader_1659349
034750/work
datashape==0.5.4
debugpy @ file:///C:/ci/debugpy_1637091961445/work
decorator @ file:///opt/conda/conda-bld/decorator_1643638310831/work
defusedxml @ file:///tmp/build/80754af9/defusedxml_1615228127516/work
diff-match-patch @
file:///Users/ktietz/demo/mc3/conda-bld/diff-match-patch_1630511840874/
work
dill @ file:///tmp/build/80754af9/dill_1623919422540/work
distributed @ file:///C:/ci/distributed_1658505715201/work
docutils @
file:///C:/Windows/TEMP/abs_24e5e278-4d1c-47eb-97b9-f761d871f482dy2vg45
0/croots/recipe/docutils_1657175444608/work
entrypoints @ file:///C:/ci/entrypoints_1649926621128/work
et-xmlfile==1.1.0
exceptiongroup==1.3.1
fastapi==0.128.8
fastjsonschema @
file:///C:/Users/BUILDE~1/AppData/Local/Temp/abs_ebruxzvd08/croots/reci
pe/python-fastjsonschema_1661376484940/work
filelock @ file:///opt/conda/conda-bld/filelock_1647002191454/work
flake8 @ file:///opt/conda/conda-bld/flake8_1648129545443/work
Flask @ file:///home/ktietz/src/ci/flask_1611932660458/work
fonttools==4.25.0
fsspec @
file:///C:/Windows/TEMP/abs_a2mhnomvfy/croots/recipe/fsspec_16599722245
40/work
future @ file:///C:/ci/future_1607568713721/work
gensim @ file:///C:/ci/gensim_1646825438310/work
gitdb==4.0.12
GitPython==3.1.46
glob2 @ file:///home/linux1/recipes/ci/glob2_1610991677669/work
greenlet @ file:///C:/ci/greenlet_1628888275363/work
```

```
h11==0.16.0
h5py @ file:///C:/ci/h5py_1659089875384/work
HeapDict @
file:///Users/ktietz/demo/mc3/conda-bld/heapdict_1630598515714/work
holoviews @
file:///C:/Windows/TEMP/abs_fb9d2988-9681-46b6-b9ab-190cfecd15d7uinyq_4
z/croots/recipe/holoviews_1658171511842/work
hvplot @
file:///C:/Windows/TEMP/abs_02zoq6lck3/croots/recipe/hvplot_16590265020
64/work
hyperlink @ file:///tmp/build/80754af9/hyperlink_1610130746837/work
idna @ file:///tmp/build/80754af9/idna_1637925883363/work
imagecodecs @
file:///C:/b/abs_948ub5byiu/croots/recipe/imagecodecs_1664562381493/wor
k
imageio @
file:///C:/Windows/TEMP/abs_24c1b783-7540-4ca9-a1b1-0e8aa8e6ae64hb79ssu
x/croots/recipe/imageio_1658785038775/work
imagesize @
file:///C:/Windows/TEMP/abs_3cecd249-3fc4-4bfc-b80b-bb227b0d701en12vqzo
t/croots/recipe/imagesize_1657179501304/work
importlib-metadata @
file:///C:/ci/importlib-metadata_1648562621412/work
incremental @ file:///tmp/build/80754af9/incremental_1636629750599/work
inflection==0.5.1
iniconfig @ file:///home/linux1/recipes/ci/iniconfig_1610983019677/work
intake @ file:///opt/conda/conda-bld/intake_1647436631684/work
intervaltree @
file:///Users/ktietz/demo/mc3/conda-bld/intervaltree_1630511889664/work
ipykernel @
file:///C:/b/abs_21ykzkm7y_/croots/recipe/ipykernel_1662361803478/work
ipython @
file:///C:/Windows/TEMP/abs_45b5zb1l7q/croots/recipe/ipython_1659529855
872/work
ipython-genutils @
file:///tmp/build/80754af9/ipython_genutils_1606773439826/work
ipywidgets @ file:///tmp/build/80754af9/ipywidgets_1634143127070/work
isort @ file:///tmp/build/80754af9/isort_1628603791788/work
itemadapter @ file:///tmp/build/80754af9/itemadapter_1626442940632/work
itemloaders @
file:///opt/conda/conda-bld/itemloaders_1646805235997/work
itsdangerous @
file:///tmp/build/80754af9/itsdangerous_1621432558163/work
```

```
jdcal @
file:///Users/ktietz/demo/mc3/conda-bld/jdcal_1630584345063/work
jedi @ file:///C:/ci/jedi_1644315428289/work
jellyfish @ file:///C:/ci/jellyfish_1647962783748/work
Jinja2 @ file:///tmp/build/80754af9/jinja2_1612213139570/work
jinja2-time @
file:///opt/conda/conda-bld/jinja2-time_1649251842261/work
jmespath @
file:///Users/ktietz/demo/mc3/conda-bld/jmespath_1630583964805/work
joblib @ file:///tmp/build/80754af9/joblib_1635411271373/work
json5 @ file:///tmp/build/80754af9/json5_1624432770122/work
jsonschema @
file:///C:/b/abs_59eyhnbyej/croots/recipe/jsonschema_1663375476535/work
jupyter @
file:///C:/Windows/TEMP/abs_56xfdi__li/croots/recipe/jupyter_1659349053
177/work
jupyter-console @
file:///opt/conda/conda-bld/jupyter_console_1647002188872/work
jupyter-server @
file:///C:/Windows/TEMP/abs_d3c42c59-765d-4f9b-9fa3-ad5b1369485611i_yua
l/croots/recipe/jupyter_server_1658754493238/work
jupyter_client @ file:///C:/ci/jupyter_client_1661836943389/work
jupyter_core @
file:///C:/b/abs_a9330r1z_i/croots/recipe/jupyter_core_1664917313457/wo
rk
jupyterlab @ file:///C:/ci/jupyterlab_1658891142428/work
jupyterlab-pygments @
file:///tmp/build/80754af9/jupyterlab_pygments_1601490720602/work
jupyterlab-server @
file:///opt/conda/conda-bld/jupyterlab_server_1644500396812/work
jupyterlab-widgets @
file:///tmp/build/80754af9/jupyterlab_widgets_1609884341231/work
keyring @ file:///C:/ci/keyring_1638531673471/work
kiwisolver @ file:///C:/ci/kiwisolver_1653292407425/work
lazy-object-proxy @ file:///C:/ci/lazy-object-proxy_1616529288960/work
libarchive-c @
file:///tmp/build/80754af9/python-libarchive-c_1617780486945/work
llvmlite==0.38.0
locket @ file:///C:/ci/locket_1652904031364/work
lxml @ file:///C:/ci/lxml_1657527445690/work
lz4 @ file:///C:/ci/lz4_1619516674350/work
Markdown @ file:///C:/ci/markdown_1614364082838/work
MarkupSafe @ file:///C:/ci/markupsafe_1621528502553/work
```

```
matplotlib @ file:///C:/ci/matplotlib-suite_1660169687702/work
matplotlib-inline @ file:///C:/ci/matplotlib-inline_1661915841596/work
mccabe==0.6.1
menuinst @
file:///C:/Users/BUILDE~1/AppData/Local/Temp/abs_455sf5o0ct/croots/reci
pe/menuinst_1661805970842/work
mistune @ file:///C:/ci/mistune_1607359457024/work
mkl-fft==1.3.1
mkl-random @ file:///C:/ci/mkl_random_1626186184308/work
mkl-service==2.4.0
mock @ file:///tmp/build/80754af9/mock_1607622725907/work
mpmath==1.2.1
msgpack @ file:///C:/ci/msgpack-python_1652329316214/work
multipledispatch @ file:///C:/ci/multipledispatch_1607574329826/work
munkres==1.1.4
mypy-extensions==0.4.3
narwhals==2.19.0
navigator-updater==0.3.0
nbclassic @ file:///opt/conda/conda-bld/nbclassic_1644943264176/work
nbclient @ file:///C:/ci/nbclient_1650290387259/work
nbconvert @ file:///C:/ci/nbconvert_1649741016669/work
nbformat @
file:///C:/b/abs_1dw90o2uqb/croots/recipe/nbformat_1663744957967/work
nest-asyncio @ file:///C:/ci/nest-asyncio_1649829929390/work
networkx @ file:///C:/ci/networkx_1657716998256/work
nltk @ file:///opt/conda/conda-bld/nltk_1645628263994/work
nose @ file:///opt/conda/conda-bld/nose_1642704612149/work
notebook @
file:///C:/Windows/TEMP/abs_79abr1_60s/croots/recipe/notebook_165908366
1851/work
numba @ file:///C:/ci/numba_1650394399948/work
numexpr @
file:///C:/Windows/Temp/abs_e2036a32-9fe9-47f3-a04c-dbb1c232ba4b334exiu
r/croots/recipe/numexpr_1656940304835/work
numpy==2.0.2
numpydoc @
file:///C:/Windows/TEMP/abs_30799058-86dd-4401-b621-d172137a4d87_ra3twm
7/croots/recipe/numpydoc_1657529873713/work
olefile @
file:///Users/ktietz/demo/mc3/conda-bld/olefile_1629805411829/work
openpyxl==3.0.10
packaging @ file:///tmp/build/80754af9/packaging_1637314298585/work
pandas==2.3.3
```

```
pandocfilters @
file:///opt/conda/conda-bld/pandocfilters_1643405455980/work
panel @ file:///C:/ci/panel_1657899702145/work
param @ file:///tmp/build/80754af9/param_1636647414893/work
paramiko @ file:///opt/conda/conda-bld/paramiko_1640109032755/work
parsel @ file:///C:/ci/parsel_1646740216444/work
parso @ file:///opt/conda/conda-bld/parso_1641458642106/work
partd @ file:///opt/conda/conda-bld/partd_1647245470509/work
pathlib @
file:///Users/ktietz/demo/mc3/conda-bld/pathlib_1629713961906/work
pathspec @
file:///C:/Windows/TEMP/abs_581d0u45mh/croots/recipe/pathspec_165962713
2171/work
patsy==0.5.2
pep8==1.7.1
pexpect @ file:///tmp/build/80754af9/pexpect_1605563209008/work
pickleshare @ file:///tmp/build/80754af9/pickleshare_1606932040724/work
Pillow==9.2.0
pkginfo @ file:///tmp/build/80754af9/pkginfo_1643162084911/work
platformdirs @
file:///C:/b/abs_73cc5cz_1u/croots/recipe/platformdirs_1662711386458/wo
rk
plotly @ file:///C:/ci/plotly_1658142442431/work
pluggy @ file:///C:/ci/pluggy_1648024580010/work
poyo @ file:///tmp/build/80754af9/poyo_1617751526755/work
prometheus-client @
file:///C:/Windows/TEMP/abs_ab9nx8qb08/croots/recipe/prometheus_client_
1659455104602/work
prompt-toolkit @
file:///tmp/build/80754af9/prompt-toolkit_1633440160888/work
Protego @ file:///tmp/build/80754af9/protego_1598657180827/work
protobuf==6.33.6
psutil @
file:///C:/Windows/Temp/abs_b2c2fd7f-9fd5-4756-95ea-8aed74d0039flsd9quf
z/croots/recipe/psutil_1656431277748/work
ptyprocess @
file:///tmp/build/80754af9/ptyprocess_1609355006118/work/dist/ptyproces
s-0.7.0-py2.py3-none-any.whl
py @ file:///opt/conda/conda-bld/py_1644396412707/work
pyarrow==21.0.0
pyasn1 @
file:///Users/ktietz/demo/mc3/conda-bld/pyasn1_1629708007385/work
pyasn1-modules==0.2.8
```

```
pycodestyle @ file:///tmp/build/80754af9/pycodestyle_1636635402688/work
pycosat==0.6.3
pycparser @ file:///tmp/build/80754af9/pycparser_1636541352034/work
pyct @ file:///C:/ci/pyct_1658488033428/work
pycurl==7.45.1
pydantic==2.13.2
pydantic_core==2.46.2
pydeck==0.9.2
PyDispatcher==2.0.5
pydocstyle @ file:///tmp/build/80754af9/pydocstyle_1621600989141/work
pyerfa @ file:///C:/ci/pyerfa_1621560974055/work
pyflakes @ file:///tmp/build/80754af9/pyflakes_1636644436481/work
Pygments @ file:///opt/conda/conda-bld/pygments_1644249106324/work
PyHamcrest @ file:///tmp/build/80754af9/pyhamcrest_1615748656804/work
PyJWT @ file:///C:/ci/pyjwt_1657511236979/work
pylint @
file:///C:/Windows/TEMP/abs_518eqlbmoo/croots/recipe/pylint_16591103542
41/work
pyls-spyder==0.4.0
PyNaCl @
file:///C:/Windows/Temp/abs_d5c3ajcm87/croots/recipe/pynacl_16596206674
90/work
pyodbc @
file:///C:/Windows/Temp/abs_61e3jz3u05/croots/recipe/pyodbc_16595138014
02/work
pyOpenSSL @ file:///opt/conda/conda-bld/pyopenssl_1643788558760/work
pyparsing @
file:///C:/Users/BUILDE~1/AppData/Local/Temp/abs_7f_7lba6rl/croots/reci
pe/pyparsing_1661452540662/work
pyrsistent @ file:///C:/ci/pyrsistent_1636093225342/work
PySocks @ file:///C:/ci/pysocks_1605307512533/work
pytest==7.1.2
python-dateutil @
file:///tmp/build/80754af9/python-dateutil_1626374649649/work
python-esios==0.2.7
python-lsp-black @
file:///tmp/build/80754af9/python-lsp-black_1634232156041/work
python-lsp-jsonrpc==1.0.0
python-lsp-server @
file:///tmp/build/80754af9/python-lsp-server_1648176833691/work
python-slugify @
file:///tmp/build/80754af9/python-slugify_1620405669636/work
python-snappy @ file:///C:/ci/python-snappy_1610133405910/work
```

```
pytz @
file:///C:/Windows/TEMP/abs_90eacd4e-8eff-491e-b26e-f707eba2cbe1ujvbhqz
1/croots/recipe/pytz_1654762631027/work
pyviz-comms @ file:///tmp/build/80754af9/pyviz_comms_1623747165329/work
PyWavelets @ file:///C:/ci/pywavelets_1648728084106/work
pywin32==302
pywin32-ctypes @ file:///C:/ci/pywin32-ctypes_1607553594546/work
pywinpty @
file:///C:/ci_310/pywinpty_1644230983541/work/target/wheels/pywinpty-2.
0.2-cp39-none-win_amd64.whl
PyYAML==6.0
pyzmq @ file:///C:/ci/pyzmq_1657615952984/work
QDarkStyle @ file:///tmp/build/80754af9/qdarkstyle_1617386714626/work
qstylizer @
file:///tmp/build/80754af9/qstylizer_1617713584600/work/dist/qstylizer-
0.1.10-py2.py3-none-any.whl
QtAwesome @ file:///tmp/build/80754af9/qtawesome_1637160816833/work
qtconsole @ file:///opt/conda/conda-bld/qtconsole_1643819126524/work
QtPy @ file:///C:/ci/qtpy_1662015096047/work
queuelib==1.5.0
regex @ file:///C:/ci/regex_1658258307256/work
requests @ file:///C:/ci/requests_1657735342357/work
requests-file @
file:///Users/ktietz/demo/mc3/conda-bld/requests-file_1629455781986/wor
k
rope @ file:///opt/conda/conda-bld/rope_1643788605236/work
Rtree @ file:///C:/ci/rtree_1618421015405/work
ruamel-yaml-conda @ file:///C:/ci/ruamel_yaml_1616016898638/work
s3transfer @ file:///C:/ci/s3transfer_1654512518418/work
scikit-image @ file:///C:/ci/scikit-image_1648214340990/work
scikit-learn @ file:///C:/ci/scikit-learn_1642617276183/work
scikit-learn-intelex==2021.20221004.171935
scipy==1.9.1
Scrapy @
file:///C:/Windows/TEMP/abs_f50e21j997/croots/recipe/scrapy_16595987071
53/work
seaborn @ file:///tmp/build/80754af9/seaborn_1629307859561/work
Send2Trash @ file:///tmp/build/80754af9/send2trash_1632406701022/work
service-identity @
file:///Users/ktietz/demo/mc3/conda-bld/service_identity_1629460757137/
work
sip==4.19.13
six @ file:///tmp/build/80754af9/six_1644875935023/work
```

```
smart-open @ file:///C:/ci/smart_open_1651235069716/work
smmap==5.0.3
sniffio @ file:///C:/ci/sniffio_1614030527509/work
snowballstemmer @
file:///tmp/build/80754af9/snowballstemmer_1637937080595/work
sortedcollections @
file:///tmp/build/80754af9/sortedcollections_1611172717284/work
sortedcontainers @
file:///tmp/build/80754af9/sortedcontainers_1623949099177/work
soupsieve @ file:///tmp/build/80754af9/soupsieve_1636706018808/work
Sphinx @ file:///C:/ci/sphinx_1657617205740/work
sphinxcontrib-applehelp @
file:///home/ktietz/src/ci/sphinxcontrib-applehelp_1611920841464/work
sphinxcontrib-devhelp @
file:///home/ktietz/src/ci/sphinxcontrib-devhelp_1611920923094/work
sphinxcontrib-htmlhelp @
file:///tmp/build/80754af9/sphinxcontrib-htmlhelp_1623945626792/work
sphinxcontrib-jsmath @
file:///home/ktietz/src/ci/sphinxcontrib-jsmath_1611920942228/work
sphinxcontrib-qthelp @
file:///home/ktietz/src/ci/sphinxcontrib-qthelp_1611921055322/work
sphinxcontrib-serializinghtml @
file:///tmp/build/80754af9/sphinxcontrib-serializinghtml_1624451540180/
work
spyder @
file:///C:/Windows/TEMP/abs_66k5aq_mkw/croots/recipe/spyder_16595998053
72/work
spyder-kernels @ file:///C:/ci/spyder-kernels_1647011408404/work
SQLAlchemy @
file:///C:/Windows/Temp/abs_f8661157-660b-49bb-a790-69ab9f3b8f7c8a8s2ps
b/croots/recipe/sqlalchemy_1657867864564/work
starlette==0.49.3
statsmodels==0.13.2
streamlit==1.50.0
sympy @ file:///C:/ci/sympy_1647853873858/work
tables==3.6.1
tabulate @ file:///C:/ci/tabulate_1657619055201/work
TBB==0.2
tblib @
file:///Users/ktietz/demo/mc3/conda-bld/tblib_1629402031467/work
tenacity==9.1.2
terminado @ file:///C:/ci/terminado_1644322780199/work
```

```
testpath @
file:///C:/Windows/TEMP/abs_23c7fa33-cbb9-46dc-b7c5-590c38e2de3d4bmbnga
l/croots/recipe/testpath_1655908553202/work
text-unidecode @
file:///Users/ktietz/demo/mc3/conda-bld/text-unidecode_1629401354553/wo
rk
textdistance @
file:///tmp/build/80754af9/textdistance_1612461398012/work
threadpoolctl @
file:///Users/ktietz/demo/mc3/conda-bld/threadpoolctl_1629802263681/wor
k
three-merge @ file:///tmp/build/80754af9/three-merge_1607553261110/work
tifffile @ file:///tmp/build/80754af9/tifffile_1627275862826/work
tinycss @ file:///tmp/build/80754af9/tinycss_1617713798712/work
tldextract @ file:///opt/conda/conda-bld/tldextract_1646638314385/work
toml @ file:///tmp/build/80754af9/toml_1616166611790/work
tomli @
file:///C:/Windows/TEMP/abs_ac109f85-a7b3-4b4d-bcfd-52622eceddf0hy332oj
o/croots/recipe/tomli_1657175513137/work
tomlkit @
file:///C:/Windows/TEMP/abs_3296qo9v6b/croots/recipe/tomlkit_1658946894
808/work
toolz @ file:///tmp/build/80754af9/toolz_1636545406491/work
tornado @ file:///C:/ci/tornado_1606924294691/work
tqdm @
file:///C:/b/abs_0axbz66qik/croots/recipe/tqdm_1664392691071/work
traitlets @ file:///tmp/build/80754af9/traitlets_1636710298902/work
Twisted @
file:///C:/Windows/Temp/abs_ccblv2rzfa/croots/recipe/twisted_1659592764
512/work
twisted-iocpsupport @
file:///C:/ci/twisted-iocpsupport_1646798932792/work
typing-inspection==0.4.2
typing_extensions==4.15.0
tzdata==2026.1
ujson @ file:///C:/ci/ujson_1657525944442/work
Unidecode @ file:///tmp/build/80754af9/unidecode_1614712377438/work
urllib3 @
file:///C:/Windows/TEMP/abs_65ynz4fdmi/croots/recipe/urllib3_1659110473
919/work
uvicorn==0.39.0
w3lib @
file:///Users/ktietz/demo/mc3/conda-bld/w3lib_1629359764703/work
```

```
watchdog @ file:///C:/ci/watchdog_1638367441841/work
wcwidth @
file:///Users/ktietz/demo/mc3/conda-bld/wcwidth_1629357192024/work
webencodings==0.5.1
websocket-client @ file:///C:/ci/websocket-client_1614804375980/work
Werkzeug @ file:///opt/conda/conda-bld/werkzeug_1645628268370/work
widgetsnbextension @
file:///C:/ci/widgetsnbextension_1644991377168/work
win-inet-pton @ file:///C:/ci/win_inet_pton_1605306162074/work
win-unicode-console==0.5
wincertstore==0.2
wrapt @
file:///C:/Windows/Temp/abs_7c3dd407-1390-477a-b542-fd15df6a24085_diwiz
a/croots/recipe/wrapt_1657814452175/work
xarray @ file:///opt/conda/conda-bld/xarray_1639166117697/work
xlrd @ file:///tmp/build/80754af9/xlrd_1608072521494/work
XlsxWriter @ file:///opt/conda/conda-bld/xlsxwriter_1649073856329/work
xlwings @
file:///C:/b/abs_41rmpiyy58/croots/recipe/xlwings_1664932818417/work
yapf @ file:///tmp/build/80754af9/yapf_1615749224965/work
zict==2.1.0
zipp @ file:///C:/ci/zipp_1652273994994/work
zope.interface @ file:///C:/ci/zope.interface_1625036252485/work
```